

Distributed Multirobot Task Assignment via Consensus ADMM

Ola Shorinwa , Ravi N. Haksar , Patrick Washington, *Graduate Student Member, IEEE*,
and Mac Schwager , *Member, IEEE*

Abstract—In this article, we present a distributed algorithm to solve a class of multirobot task assignment problems. We formulate task assignment as a mathematical optimization and solve for optimal solutions with a variant of the consensus alternating direction method of multipliers (C-ADMM). We provide C-ADMM-based algorithms for both the primal and dual problem formulations and show the advantages of each form depending on the problem specifics. In our algorithm, each robot solves a series of local optimization problems and communicates the results to its local neighbors, ultimately converging to an optimal task assignment in problems with linear objective functions and an optimal solution of the relaxed problem in convex problems. While many other distributed algorithms require a central station for their implementation, in our algorithm, each robot only communicates with its one-hop neighbors. In linear task assignment problems, our algorithm converges to the optimal task assignment, unlike many other distributed algorithms for this problem, which yield suboptimal solutions. We demonstrate our algorithms in task assignment problems over a variety of communication network topologies, where we show that our inexact dual algorithm is at least 60% faster than other distributed algorithms, which produce an optimal task assignment. In addition, our dual algorithm attains a 69% speedup in computation time compared to a notable distributed variant of the Hungarian method (Chopra et al., 2017). We also apply our algorithm to a multi-unmanned-aerial-vehicle persistent surveillance problem, showing its suitability for problems involving periodic task assignments.

Index Terms—Distributed mathematical programming, distributed optimization, distributed task assignment, multirobot systems.

Manuscript received 29 August 2022; accepted 9 November 2022. Date of publication 2 January 2023; date of current version 7 June 2023. This work was supported in part by the Defense Advanced Research Projects Agency's Young Faculty Award D18AP00064, in part by the National Science Foundation's National Robotics Initiative under Award 1830402 and Award 1925030, in part by the NASA University Leadership Initiative under Grant 80NSSC20M0163, and in part by the Toyota Research Institute. This article solely reflects the opinions and conclusions of its authors and not any NASA entity. This paper was recommended for publication by Associate Editor P. Tokekar and Editor P. Robuffo Giordano upon evaluation of the reviewers' comments. (*Corresponding author: Ola Shorinwa.*)

Ola Shorinwa is with the Department of Mechanical Engineering, Stanford University, Stanford, CA 94305 USA (e-mail: shorinwa@stanford.edu).

Ravi N. Haksar is with the Nuro, Mountain View, CA 94043 USA (e-mail: rhaksar@nuro.ai).

Patrick Washington and Mac Schwager are with the Department of Aeronautics and Astronautics Engineering, Stanford University, Stanford, CA 94305 USA (e-mail: phw@stanford.edu; schwager@stanford.edu).

This article has supplementary material provided by the authors and color versions of one or more figures available at <https://doi.org/10.1109/TRO.2022.3228132>.

Digital Object Identifier 10.1109/TRO.2022.3228132

I. INTRODUCTION

MULTIROBOT task assignment problems arise in a variety of applications, including disaster and rescue operations [2], persistent surveillance [3], [4], package delivery [5], [6], and transportation [7], [8], [9], where the deployment of multiple robots allows for the completion of several tasks concurrently. In many situations, the robots possess heterogeneous capabilities, with certain robots better suited for specific tasks. In realistic settings, each robot has access to only its local data and communication links to local neighbors, and as a result, computing the optimal task assignment requires collaboration among all robots. Centralized methods collate all the information available to each robot at a central station, which computes the optimal task assignment. This approach presents significant computational and communication overhead, especially in problems with a large number of robots, and introduces a brittle single point of failure in the central station. In this article, we introduce distributed algorithms for the multirobot task assignment problem, where each robot computes an optimal task assignment in problems with linear objective functions and, more generally, an optimal solution of the relaxed task assignment problem locally, overcoming these challenges.

We formulate the multirobot task assignment problem as a mathematical program, considering both its primal and dual forms, noting that these convex formulations yield the optimal task assignment [10] when the objective function consists of a sum of linear functions. This article has evolved from our earlier work [11], in which we derived specialized primal and inexact dual algorithms for the multirobot task assignment problem, demonstrating its application to task assignment problems with linear objective functions. In this article, we extend [11] by presenting a distributed exact dual algorithm, in addition to providing a more thorough analysis of our distributed algorithms, which include a multirobot primal algorithm for the task assignment problem (MUR-TAP), a multirobot exact dual algorithm for the task assignment problem (MURD-TAP), and a multirobot inexact dual algorithm for the task assignment problem (MURID-TAP). Collectively, we refer to our distributed algorithms as the *MUR Family* of algorithms for *task assignment* problems. Our distributed task assignment algorithms apply to task assignment problems with convex (linear or nonlinear) objective functions and affine constraints a broader class of task assignment problems than those previously treated in the literature, which subsumes the classical task assignment problem with linear objective functions.

Our distributed algorithms are based on the alternating direction method of multipliers (ADMM), which builds upon the method of multipliers and the dual decomposition method, blending the benefits of both methods to provide improved convergence for a broader class of problems. While the ADMM typically requires a central station for updating the dual variables [12], [13], we utilize a fully distributed variant of ADMM, consensus ADMM [14], which imposes consensus constraints on the local optimization variables of each robot, ensuring that all the robots arrive at the same optimal solution. Other fully distributed variants of the ADMM exist [15], [16]; however, these methods place stringent conditions on the problem.

In our algorithms, each robot solves a local optimization problem iteratively, while communicating with its neighbors, to compute an optimal solution of the mathematical program. We assume that each robot communicates with only its one-hop neighbors over a point-to-point communication network. Furthermore, our algorithm does not require strategic data flow during the communication process. Although each robot has access to only its local problem data, all the robots compute the same optimal solution, without relying on a central station at any point. Consequently, our algorithms offer stronger protection against adversarial attacks, eliminating vulnerabilities associated with a single point of failure. Moreover, the iterates in our algorithms converge to the optimal solution of the relaxed task assignment problem, provided that the objective function of the problem is proper and convex, and the Lagrangian of the problem has a saddle point (standard technical conditions for solving convex optimization problems), unlike many existing distributed methods that produce suboptimal solutions. As with other distributed methods, our algorithms provide better privacy by minimizing the amount of data shared by each robot, as each robot does not share its local objective and constraint functions data with other robots.

We demonstrate the performance of our task assignment algorithms in comparison to other distributed task assignment algorithms, showing the balanced tradeoff provided by our algorithms with respect to communication and computation overhead. Particularly, MURID-TAP is about 60% faster compared to the best competing distributed algorithm [17], among distributed algorithms that produce an optimal task assignment, in terms of the cumulative computation time required by each robot to compute an optimal task assignment. In comparison to a distributed variant of the widely used Hungarian method for task assignment [1], MURD-TAP provides about a 69% speedup in computation time per robot at the expense of a greater number of communication rounds. Furthermore, we demonstrate the application of our distributed algorithm to persistent surveillance problems, where our algorithm produces the optimal task assignment at each assignment episode. In addition, we show the amenability of our algorithms to problems with different communication network topologies and, further, highlight the versatility of our algorithms in multirobot problems with constraints on the availability of different resources, including computational, communication, and data storage resources.

II. RELATED WORKS

The task assignment problem, a canonical optimization scenario, with applications extending beyond the multirobot systems domain to task scheduling and operations management, is widely solved using the Hungarian algorithm [18]—a centralized algorithm that requires the knowledge of the costs of all the possible robot and task pairs. The need for global access to the costs of all the robot and task pairs renders the algorithm unsuitable for multirobot problems, where each robot only has access to its local costs for each task. Dispersing information on the costs among all robots introduces computational and communication overhead, which degrades the efficiency of the algorithm. A few distributed variants of the Hungarian algorithm overcome these challenges [1], [19] and still provide the optimal task assignment. As with the centralized Hungarian algorithm, these distributed variants only apply to problems with linear objective cost functions, limiting the scope of multirobot task assignment problems that can be solved using these methods. In contrast, our methods can be used for problems with general convex objectives.

Another popular approach, auction methods, involves a market construct where the robots negotiate with their neighbors to compute an assignment that minimizes their costs while ensuring that no task remains unassigned. Typically, a centralized auctioneer manages the auctions, receiving bids from the robots before assigning the task to the winning bidder [20], [21], [22], [23]. Consequently, these methods require all robots to communicate with the auctioneer [24]. Other methods designate an auctioneer from the group of robots, which also doubles as a bidder [25], [26], with the auction process only being executed when the robots are within the range of their neighbors for spatially distributed tasks [27], [28]. In other methods, each robot serves as an auctioneer, updating the assignment of all tasks iteratively upon receiving bids from its neighbors until all the tasks are assigned [29], [30], [31], [32], [33], [34]. Despite the relative simplicity of auction methods, these methods do not generally provide an optimal assignment of all tasks [35], [36]. However, the auction method in [17], which involves a conflict resolution procedure for robots assigned to the same task, results in an optimal assignment. Unlike auction or market methods, our method produces the optimal task assignment without relying on any designated auctioneer.

The task assignment problem can be formulated as a mixed-integer optimization problem with binary assignment variables. As such, mixed-integer linear programming (MILP) methods apply to these problems with linear objective functions. The MILP methods in [37], [38], [39], and [40] solve the task assignment problem at a central station, which collates all the relevant local information available to each robot. Some distributed MILP methods employ dual decomposition in solving the task assignment problem [41] without requiring each robot to communicate its local objective functions. However, these methods still require a central station for updating the dual variables after the local primal updates performed by each robot. Other methods utilize a primal decomposition approach to compute an approximate solution to the task assignment problem [42]. In [43], each

robot computes its assignment from the dual formulation of the task assignment problem by solving MILP subproblems iteratively using the branch and bound procedure. Other MILP methods utilize a distributed cutting-plane procedure to solve the task assignment problem but require the objective function to be integer-valued [44], introducing an additional limitation on the class of task assignment problems suitable for these methods.

Some distributed methods consider a convex relaxation of the task assignment problem to a linear program with real-valued variables over the unit simplex, producing the same optimal solution as the original mixed-integer problem. These distributed methods only apply to task assignment problems with linear objective functions. In these methods, the robots compute the optimal assignment using a distributed simplex algorithm [10] or dual decomposition [45]. The surveys [46] and [47] provide an overview of methods for the multirobot task assignment problem, noting the conditions required for the implementation of these methods. We consider a convex relaxation of the task assignment problem. However, we do not limit the scope of our algorithms to problems with linear objective functions. Rather, we derive algorithms that equally apply to problems with nonlinear objective functions. In addition, our algorithms do not require any stringent condition on the nature of the objective function or the topology of the communication network.

III. CONTRIBUTIONS

We summarize our contributions as follows.

- 1) We provide distributed algorithms for multirobot task assignment problems (MUR-TAP and MURD-TAP), where each robot computes the optimal solution of the relaxed task assignment problem, which corresponds to the optimal task assignment in problems with linear objective functions, by solving a sequence of local optimization problems. In our algorithms, each robot communicates with other neighboring robots over a local network, without any stringent conditions on the topology of the communication network.
- 2) In certain problems, multirobot systems operate with limited access to computational resources. As a result, solving the optimization problems arising in MUR-TAP and MURD-TAP might be computationally demanding for these systems. We derive MURID-TAP for these specific situations. MURID-TAP enables each robot to compute an optimal solution of the relaxed task assignment problem via closed-form updates, without having to resort to a nested iterative optimization method to solve its local optimization problems.
- 3) Compared to existing distributed algorithms for the task assignment problem, which produce an optimal task assignment [1], [17], MURID-TAP provides at least 60% faster performance in computing an optimal task assignment, with respect to the cumulative computation time per robot.

Organization: The rest of this article is organized as follows. We present the classical task assignment problem with linear objective functions in Section IV before providing a more general formulation for problems with nonlinear objective functions and affine constraints, noting its distributed nature over a network of robots where each robot has access to only its local objective and constraint functions. We derive task assignment problems in Section VI. We demonstrate our distributed multirobot task assignment algorithms in Section VII, examining their performance in comparison to other distributed task assignment methods. Finally, Section VIII concludes this article.

IV. PROBLEM FORMULATION

We consider the classical multirobot task assignment problem where we seek an optimal assignment of N robots to N tasks, described by the optimization problem

$$\begin{aligned}
 & \underset{x}{\text{minimize}} && \sum_{i=1}^N c_i^T x_i \\
 & \text{subject to} && \sum_{i=1}^N x_i = \mathbf{1}_N \\
 & && \mathbf{1}_N^T x_i = 1 \quad \forall i \in \mathcal{V} \\
 & && x_{i,\tau} \in \{0, 1\} \quad \forall \tau \in \mathcal{T}, \forall i \in \mathcal{V}
 \end{aligned} \tag{1}$$

where $x_i \in \mathbb{R}^N$ denotes the optimization variable of robot i , with component τ of x_i indicating if a robot is assigned to task τ and $x = [x_1^T, \dots, x_N^T]^T \in \mathbb{R}^{N^2}$. In addition, $c_i \in \mathbb{R}^N$ denotes the objective cost vector of robot i . We denote the set of all tasks by $\mathcal{T} = \{1, \dots, N\}$. The optimization problem in (1) represents an integer optimization problem from the binary constraints on x_i . Solving this combinatorial optimization problem proves challenging, with typical algorithms often resorting to branch-and-bound methods, which fail to scale to problems with large numbers of robots. Generally, existing methods solve a relaxation of the task assignment problem, which replaces the binary constraints with box constraints on x_i , $\forall i \in \mathcal{V}$. The relaxed problem is given by

$$\begin{aligned}
 & \underset{x}{\text{minimize}} && \sum_{i=1}^N c_i^T x_i \\
 & \text{subject to} && \sum_{i=1}^N x_i = \mathbf{1}_N \\
 & && \mathbf{1}_N^T x_i = 1 \quad \forall i \in \mathcal{V} \\
 & && 0 \leq x_{i,\tau} \leq 1 \quad \forall \tau \in \mathcal{T}, \forall i \in \mathcal{V}
 \end{aligned} \tag{2}$$

with x_i constrained to lie between 0 and 1. Given a linear objective function, an optimal solution of the problem (2) always occurs at a vertex of the feasible set, since the relaxed optimization problem has a bounded feasible set, although the optimal solution might not be unique. As a result, an integer-valued

solution can always be obtained from an optimal solution of (2). Moreover, this solution corresponds to an optimal solution of the integer optimization problem in (1).

In more complex problems, the cost incurred by each robot in performing a task is better captured by nonlinear objective functions, e.g., in problems with strictly convex cost functions. In addition, many task assignment problems often involve additional constraints arising from individual preferences and task priorities. As a result, we extend the problem formulation in (1) to consider a broader class of task assignment problems with N robots, m tasks, convex (linear/nonlinear) objective functions, and affine constraints, given by

$$\begin{aligned} & \underset{x}{\text{minimize}} \quad \sum_{i=1}^N f_i(x_i) \\ & \text{subject to} \quad \sum_{i=1}^N x_i \geq \mathbf{1}_m \\ & \quad \mathbf{1}_m^\top x_i = 1 \quad \forall i \in \mathcal{V} \\ & \quad x_{i,\tau} \in \{0, 1\} \quad \forall \tau \in \mathcal{T}, \forall i \in \mathcal{V} \end{aligned} \quad (3)$$

where $x_i \in \mathbb{R}^m$ denotes the optimization variable of robot i , $x = [x_1^\top, \dots, x_N^\top]^\top \in \mathbb{R}^{Nm}$, $f_i(x) : \mathbb{R}^m \rightarrow \mathbb{R}$ denotes the local convex objective function of robot i , and $\mathcal{T} = \{1, \dots, m\}$. We note that solving the task assignment problem in (3) is even more challenging compared to solving (1), particularly via a distributed approach. Consequently, we consider a relaxation of (3), similar to (2). The relaxed problem is given by

$$\begin{aligned} & \underset{x}{\text{minimize}} \quad \sum_{i=1}^N f_i(x_i) \\ & \text{subject to} \quad \sum_{i=1}^N x_i \geq \mathbf{1}_m \\ & \quad \mathbf{1}_m^\top x_i = 1 \quad \forall i \in \mathcal{V} \\ & \quad 0 \leq x_{i,\tau} \leq 1 \quad \forall \tau \in \mathcal{T}, \forall i \in \mathcal{V} \end{aligned} \quad (4)$$

where x_i lies between 0 and 1. The formulation in (4) encompasses task assignment problems with an unequal number of robots and tasks, where $N \neq m$. We note that the optimal solution of (4) might not be integer-valued. In these cases, heuristics for obtaining a binary assignment can be employed [48], [49]. However, if an integer optimal solution is obtained for (4), then this solution is optimal for the original integer optimization problem.

V. PRELIMINARIES

We represent the robots as nodes in an undirected communication graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ with a set of vertices $\mathcal{V} = \{1, \dots, N\}$ and a set of edges $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$. An edge (i, j) exists in $\mathcal{E}$ if robots i and j share a communication link. To ensure that all robots compute the same solution, we make the following assumptions on the communication graph between the robots.

Assumption 1: The communication graph $\mathcal{G}$ is connected.

This assumption indicates that a communication path exists between any pair of robots, possibly involving multiple hops along the edges in $\mathcal{E}$.

We denote the $n \times n$ identity matrix as I_n and the vector of all ones as $\mathbf{1}_n$. We interpret box constraints on vector-valued variables elementwise. We provide the following definition of a proper, closed, and convex function, in addition to the definition of a coercive function, before introducing the next assumption.

Definition 1: A function $f : \mathbb{R}^n \rightarrow \mathbb{R} \cup \{+\infty\}$ is proper if it does not attain a value of $+\infty$ everywhere, i.e., there exists $x \in \mathbb{R}^n$ such that $f(x) \in \mathbb{R}$. Furthermore, f is closed if its epigraph $\text{epi}(f) = \{(x, t) \in \mathbb{R}^n \times \mathbb{R} \mid x \in \mathbb{R}^n, t \geq f(x)\}$ is closed.

Definition 2: A function $f : \mathbb{R}^n \rightarrow \mathbb{R} \cup \{+\infty\}$ is convex if for any $x \in \mathbb{R}^n$, $y \in \mathbb{R}^n$, and $t \in \mathbb{R}$ with $t \in [0, 1]$

$$f(tx + (1-t)y) \leq tf(x) + (1-t)f(y). \quad (5)$$

Definition 3: A continuous function $f : \mathbb{R}^n \rightarrow \mathbb{R} \cup \{+\infty\}$ is coercive if

$$\lim_{\|x\| \rightarrow \infty} f(x) = +\infty \quad (6)$$

i.e., for any constant $L > 0$, there exists a constant $R_L > 0$ such that $\|f(x)\| > L$ when $\|x\| > R_L$.

We assume that the Lagrangian of the task assignment problem in (4) has a saddle point.

Assumption 2: The objective function in (4) is proper, closed, and convex, and a saddle point exists for the Lagrangian of the task assignment problem.

This assumption is standard in many papers in constrained optimization, although it is often not stated in many of these papers. The Lagrangian of the task assignment problem has a saddle point if the task assignment problem is feasible. As such, Assumption 2 is not restrictive, as it is satisfied in almost all practical cases. This assumption enables us to compute a primal-dual solution of the problem in (4). Furthermore, this assumption indicates that strong duality holds with equality between the optimal primal objective value and the optimal dual objective value.

VI. DISTRIBUTED ALGORITHMS FOR MULTIROBOT TASK ASSIGNMENT PROBLEMS

In the following discussion, we derive distributed algorithms for the multirobot task assignment problem (4), considering its primal and dual forms.

A. Distributed Primal Algorithm

We derive a distributed algorithm for the task assignment problem in (4). To obtain a concise formulation, we group all the constraints in (4) into a single affine constraint, representing the optimization problem as

$$\begin{aligned} \mathcal{P} : \quad & \underset{x}{\text{minimize}} \quad \sum_{i=1}^N f_i(x) \\ & \text{subject to} \quad A_i x \leq b_i \quad \forall i \in \mathcal{V} \end{aligned} \quad (7)$$

where

$$A_i = \begin{bmatrix} -\check{P} \\ \check{Q}_i \\ -\check{Q}_i \\ -I_{Nm} \\ I_{Nm} \end{bmatrix}, \quad b_i = \begin{bmatrix} -\mathbf{1}_m \\ 1 \\ -1 \\ \mathbf{0}_{Nm} \\ \mathbf{1}_{Nm} \end{bmatrix} \quad (8)$$

and $\check{P} \in \mathbb{R}^{m \times Nm}$ represents a horizontal block matrix consisting of N identity matrices (I_m) concatenated horizontally, and $\check{Q}_i \in \mathbb{R}^{1 \times Nm}$ represents a horizontal block matrix of all zeros except the i th horizontal block component, which is set to $\mathbf{1}_m^T$. Although the local objective function of robot i only depends on x_i , which is a component of x , we express it in a more general form in (7), for simplicity.

We express the problem in (7) in its distributed form by introducing local copies of the optimization variables maintained by each robot. Robot i maintains $x_i \in \mathbb{R}^{Nm}$, representing a copy of x , with the resulting distributed form of the problem given by

$$\begin{aligned} & \underset{x}{\text{minimize}} \quad \sum_{i=1}^N f_i(x_i) \\ & \text{subject to} \quad A_i x_i \leq b_i \quad \forall i \in \mathcal{V} \\ & \quad \quad \quad x_i = x_j \quad \forall j \in \mathcal{N}_i, \forall i \in \mathcal{V} \end{aligned} \quad (9)$$

where $x = [x_i^T, \forall i \in \mathcal{V}]^T$ denotes the vertical concatenation of the optimization variables of all robots. We introduce the equality constraint between the optimization variables of neighboring robots to ensure agreement between all robots.

Proposition 1: The distributed problem in (9) is equivalent to the mathematical programming problem in (7) with the same optimal solution and optimal objective value.

Proof: From the equality constraints in (9), all robots compute the same solution for their local optimization variables since the communication graph $\mathcal{G}$ is connected. Consequently, we can replace the local optimization variable of each robot with a common variable $\tilde{x}$. The resulting optimization problem has the same objective function and feasible set as the problem in (7). Hence, both the problems have the same optimal solution and optimal objective value. $\square$

We derive a distributed method for solving the mathematical program in (9) using consensus ADMM [14]. We introduce the local slack variables $\alpha_{ij} \in \mathbb{R}^{Nm}$ and $\beta_{ij} \in \mathbb{R}^{Nm}$ into the equality constraints between robots i and j in (9), expressing the problem as

$$\begin{aligned} & \underset{x, \sigma}{\text{minimize}} \quad \sum_{i=1}^N f_i(x_i) \\ & \text{subject to} \quad A_i x_i \leq b_i \quad \forall i \in \mathcal{V} \\ & \quad \quad \quad x_i = \alpha_{ij} \quad \forall j \in \mathcal{N}_i, \forall i \in \mathcal{V} \\ & \quad \quad \quad x_j = \beta_{ij} \quad \forall j \in \mathcal{N}_i, \forall i \in \mathcal{V} \\ & \quad \quad \quad \alpha_{ij} = \beta_{ij} \quad \forall j \in \mathcal{N}_i, \forall i \in \mathcal{V} \end{aligned} \quad (10)$$

where robot i optimizes over α_{ij} , robot j optimizes over β_{ij} , and $\sigma \in \mathbb{R}^{2|\mathcal{E}|Nm}$ denotes the vertical concatenation of all the slack

variables. With the slack variables, the augmented Lagrangian of (9) can be expressed as

$$\begin{aligned} \mathcal{L}_a^p(x, \sigma, u, v) = & \sum_{i=1}^N f_i(x_i) \\ & + \sum_{(i,j) \in \mathcal{E}} \left(u_{ij}^T (x_i - \alpha_{ij}) + v_{ij}^T (x_j - \beta_{ij}) \right) \\ & + \frac{\rho}{2} \sum_{(i,j) \in \mathcal{E}} \left(\|x_i - \alpha_{ij}\|_2^2 + \|x_j - \beta_{ij}\|_2^2 \right) \end{aligned} \quad (11)$$

where $u_{ij} \in \mathbb{R}^{Nm}$ and $v_{ij} \in \mathbb{R}^{Nm}$ denote the Lagrange multipliers for the equality constraints between the optimization variables of robots i and j , respectively. We have not relaxed the affine inequality constraint in (9) but rather enforce that the local primal variable of each robot satisfies its local inequality constraint. Likewise, we enforce the constraint $\alpha_{ij} = \beta_{ij}$. The augmented Lagrangian includes a penalty on the violation of the equality constraints, with the contribution of this violation determined by the parameter $\rho \in \mathbb{R}$. In the ADMM, the primal variables are updated iteratively as the minimizers of the augmented Lagrangian using the Lagrange multipliers at the previous iteration before updating the Lagrange multipliers through gradient ascent on the augmented Lagrangian.

The update procedure for the slack variable consists of minimizing a strongly convex quadratic problem with a closed-form solution, given by

$$\alpha_{ij}^{k+1} = \beta_{ij}^{k+1} = \frac{1}{2} (x_i^{k+1} + x_j^{k+1}) \quad (12)$$

if the Lagrange multipliers are initialized with $u_{ij}^0 = v_{ij}^0 = 0$ (refer to [14] for additional details). Similarly, the update procedures of the Lagrange multipliers simplify to

$$\begin{aligned} u_{ij}^{k+1} &= u_{ij}^k + \frac{\rho}{2} (x_i^{k+1} - x_j^{k+1}) \\ v_{ij}^{k+1} &= v_{ij}^k + \frac{\rho}{2} (x_j^{k+1} - x_i^{k+1}) \end{aligned} \quad (13)$$

at iteration k . With the specified initialization of the Lagrange multipliers, $u_{ij}^k = -v_{ij}^k$ at each iteration k . To update its primal variable x_i at iteration k , robot i solves the minimization problem

$$\begin{aligned} & \underset{x_i}{\text{minimize}} \quad \left\{ f_i(x_i) + \sum_{j \in \mathcal{N}_i} (u_{ij}^k + v_{ji}^k)^T x_i \right. \\ & \quad \quad \quad \left. + \rho \sum_{j \in \mathcal{N}_i} \left\| x_i - \frac{x_i^k + x_j^k}{2} \right\|_2^2 \right\} \\ & \text{subject to} \quad A_i x_i \leq b_i \end{aligned} \quad (14)$$

which simplifies to

$$\begin{aligned} & \underset{x_i}{\text{minimize}} \quad \left\{ f_i(x_i) + q_i^k{}^T x_i + \rho \sum_{j \in \mathcal{N}_i} \left\| x_i - \frac{x_i^k + x_j^k}{2} \right\|_2^2 \right\} \\ & \text{subject to} \quad A_i x_i \leq b_i \end{aligned} \quad (15)$$

Algorithm 1: Distributed Multirobot Algorithm for Task Assignment Problems (MUR-TAP).

Initialization:

$$\mathbf{x}_i^0 \in \mathbb{R}^n \quad \forall i \in \mathcal{V}$$

$$q_i^0 \leftarrow 0 \quad \forall i \in \mathcal{V}$$

$$k \leftarrow 0$$

do in parallel $\forall i \in \mathcal{V}$

$$\mathbf{x}_i^{k+1} \leftarrow \text{Procedure (15)}$$

$$q_i^{k+1} \leftarrow \text{Procedure (17)}$$

$$k \leftarrow k + 1$$

while not converged or stopping criterion is not met;

where

$$q_i^k = \sum_{j \in \mathcal{N}_i} (u_{ij}^k + v_{ji}^k) \quad (16)$$

by combining the local Lagrange multipliers of robot i into $q_i \in \mathbb{R}^n$. Subsequently, robot i updates q_i using

$$q_i^{k+1} = q_i^k + \rho \sum_{j \in \mathcal{N}_i} (\mathbf{x}_i^{k+1} - \mathbf{x}_j^{k+1}) \quad (17)$$

from the update procedures in (13).

Algorithm 1 outlines our distributed primal algorithm for the task assignment problem. Each robot does not compute the slack variables α and β and the Lagrange multipliers u and v .

As noted in Algorithm 1, robot i computes its local primal variable $\mathbf{x}_i$ along with its Lagrange multiplier q_i . Our algorithm for (9), MUR-MP, does not require a feasible initialization of the local primal variables of each robot. Upon updating its local variables, each robot communicates its primal variables with its neighbors.

Theorem 1 (Convergence of $\{\mathbf{x}^k\}$): The sequence of primal variables $\{\mathbf{x}_i^k\}$ of robot i converges to the optimal solution $\mathbf{x}^*$ of the mathematical program in (7), $\forall i \in \mathcal{V}$.

Proof: We provide the proof in Appendix A. $\square$

B. Distributed Dual Algorithm

We note that the MUR-TAP algorithm requires robot i to maintain a copy of the entire task assignment problem variable $\mathbf{x} \in \mathbb{R}^{Nm}$, instead of only maintaining its local task assignment variable $\mathbf{x}_i \in \mathbb{R}^n$, which can result in unnecessary computation overhead. As such, we derive a distributed algorithm for task assignment problems by considering the dual formulation of the problem, where robot i maintains only its task assignment variable $\mathbf{x}_i$, without maintaining the task assignment variables of other robots. First, we express the task assignment problem in (4) as

$$\begin{aligned} \mathcal{D}: \quad & \underset{\mathbf{x}}{\text{minimize}} \quad \sum_{i=1}^N f_i(\mathbf{x}_i) \\ & \text{subject to} \quad \sum_{i=1}^N (A_i \mathbf{x}_i - \mathbf{b}_i) \leq 0 \end{aligned}$$

$$\mathbf{x}_i \geq 0 \quad \forall i \in \mathcal{V} \quad (18)$$

where

$$A_i = \begin{bmatrix} -I_m \\ G_i \\ -G_i \\ H_i \end{bmatrix}, \quad b_i = \begin{bmatrix} -\frac{1}{N} \mathbf{1}_m \\ \frac{1}{N} \mathbf{1}_N \\ -\frac{1}{N} \mathbf{1}_N \\ \check{v}_i \end{bmatrix} \quad (19)$$

$A_i \in \mathbb{R}^{e \times m}$, with $e = m + 2N + Nm$, and $G_i \in \mathbb{R}^{N \times m}$ has all its entries set to zero except the i th row of G_i , which is set to $\mathbf{1}_m^T$. Likewise, $H_i \in \mathbb{R}^{Nm \times Nm}$ represents a block diagonal matrix of all zeros with the i th block set to I_m , while $\check{v}_i \in \mathbb{R}^{Nm}$ represents a block column vector of all zeros with the i th block set to $\mathbf{1}_m$.

The Lagrangian of the mathematical program in (18) is given by

$$\mathcal{L}(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^N f_i(\mathbf{x}_i) + \tilde{\mathbf{y}}^T \sum_{i=1}^N (A_i \mathbf{x}_i - \mathbf{b}_i) \quad (20)$$

where $\tilde{\mathbf{y}} \in \mathbb{R}^e$ denotes the dual variable for the separable coupling constraint in (18), with $\tilde{\mathbf{y}} \geq 0$.

From the Lagrangian, we obtain the dual function associated with (18), given by

$$g(\tilde{\mathbf{y}}) = \inf_{\mathbf{x}} \left\{ \sum_{i=1}^N (f_i(\mathbf{x}_i) + \tilde{\mathbf{y}}^T (A_i \mathbf{x}_i - \mathbf{b}_i)) \right\} \quad (21)$$

involving the minimization of the Lagrangian $\mathcal{L}(\cdot)$ over $\mathbf{x}$. The dual problem associated with the mathematical program $\mathcal{D}$ in (18) consists of maximizing the dual function with respect to the dual variable $\tilde{\mathbf{y}}$, resulting in the problem

$$\begin{aligned} & \underset{\tilde{\mathbf{y}}}{\text{maximize}} \quad g(\tilde{\mathbf{y}}) \\ & \text{subject to} \quad \mathbf{y} \geq 0 \end{aligned} \quad (22)$$

which simplifies to

$$\begin{aligned} & \underset{\tilde{\mathbf{y}}}{\text{maximize}} \quad \sum_{i=1}^N \inf_{\mathbf{x}_i} (f_i(\mathbf{x}_i) + \tilde{\mathbf{y}}^T (A_i \mathbf{x}_i - \mathbf{b}_i)) \\ & \text{subject to} \quad \tilde{\mathbf{y}} \geq 0 \end{aligned} \quad (23)$$

from separability of the objective function in (21). Using Fenchel conjugates, the dual problem reduces to

$$\begin{aligned} & \underset{\tilde{\mathbf{y}}}{\text{maximize}} \quad \sum_{i=1}^N (-f_i^*(-A_i^T \tilde{\mathbf{y}}) - \tilde{\mathbf{y}}^T \mathbf{b}_i) \\ & \text{subject to} \quad \tilde{\mathbf{y}} \geq 0 \end{aligned} \quad (24)$$

where $f_i^*(\tilde{\mathbf{y}}) = \sup_{\mathbf{x}_i} \{\tilde{\mathbf{y}}^T \mathbf{x}_i - f_i(\mathbf{x}_i)\}$ denotes the Fenchel conjugate of the local objective function of robot i , which highlights the separable structure of the dual optimization problem.

For a distributed approach to solving (24), we assign local dual variables to each robot, with an equality constraint ensuring that all robots compute the same dual variables. With this approach, robot i maintains only its local dual variable $\tilde{\mathbf{y}}_i$, a copy of $\tilde{\mathbf{y}}$. We

express (24) in its distributed form as

$$\begin{aligned} & \underset{\tilde{\mathbf{y}}}{\text{maximize}} \quad \sum_{i=1}^N (-f_i^*(-A_i^T \tilde{\mathbf{y}}_i) - \tilde{\mathbf{y}}_i^T \mathbf{b}_i) \\ & \text{subject to} \quad \tilde{\mathbf{y}}_i \geq 0 \quad \forall i \in \mathcal{V} \\ & \quad \tilde{\mathbf{y}}_i = \tilde{\mathbf{y}}_j \quad \forall j \in \mathcal{N}_i, \forall i \in \mathcal{V} \end{aligned} \quad (25)$$

with the equality constraints enforced between neighboring robots, as defined by the communication graph $\mathcal{G}$. We denote the vertical concatenation of the dual variables of all robots as $\tilde{\mathbf{y}} = [\tilde{\mathbf{y}}_i^T, \forall i \in \mathcal{V}]^T$.

Proposition 2: The dual optimization problem in (25) has the same optimal solution and optimal objective value as the problem in (24).

Proof: The proof of equivalence between both problems follows along the same lines as in Proposition (1). $\square$

Following the same procedure utilized in Section VI-A, we derive a method for solving (25) where each robot computes its dual variables locally. We introduce local slack variables to allow for local computations by each robot, with the augmented Lagrangian of (25) given by

$$\begin{aligned} \mathcal{L}_a^d(\tilde{\mathbf{y}}, \gamma, \zeta, p, w) = & \sum_{i=1}^N (-f_i^*(-A_i^T \tilde{\mathbf{y}}_i) - \tilde{\mathbf{y}}_i^T \mathbf{b}_i) \\ & - \sum_{(i,j) \in \mathcal{E}} (p_{ij}^T (\tilde{\mathbf{y}}_i - \gamma_{ij}) + w_{ij}^T (\tilde{\mathbf{y}}_j - \zeta_{ij})) \\ & - \frac{\rho}{2} \sum_{(i,j) \in \mathcal{E}} (\|\tilde{\mathbf{y}}_i - \gamma_{ij}\|_2^2 + \|\tilde{\mathbf{y}}_j - \zeta_{ij}\|_2^2) \end{aligned} \quad (26)$$

with the Lagrange multipliers $p_{ij} \in \mathbb{R}^e$ and $w_{ij} \in \mathbb{R}^e$ and slack variables $\gamma_{ij} \in \mathbb{R}^e$ and $\zeta_{ij} \in \mathbb{R}^e$. Likewise, we enforce that $\tilde{\mathbf{y}}_i$ satisfies the affine inequality constraint in (25) for nonnegative dual variables and the constraint $\gamma_{ij} = \zeta_{ij}$.

Robot i computes $\tilde{\mathbf{y}}_i$ from the optimization problem

$$\begin{aligned} & \underset{\tilde{\mathbf{y}}_i}{\text{maximize}} \quad \left\{ -f_i^*(-A_i^T \tilde{\mathbf{y}}_i) - \tilde{\mathbf{y}}_i^T \mathbf{b}_i - \sum_{j \in \mathcal{N}_i} (p_{ij}^k + w_{ji}^k)^T \tilde{\mathbf{y}}_i \right. \\ & \quad \left. - \rho \sum_{j \in \mathcal{N}_i} \left\| \tilde{\mathbf{y}}_i - \frac{\tilde{\mathbf{y}}_i^k + \tilde{\mathbf{y}}_j^k}{2} \right\|_2^2 \right\} \\ & \text{subject to} \quad \tilde{\mathbf{y}}_i \geq 0 \end{aligned} \quad (27)$$

which simplifies to

$$\begin{aligned} & \underset{\tilde{\mathbf{y}}_i}{\text{maximize}} \quad \left\{ -f_i^*(-A_i^T \tilde{\mathbf{y}}_i) - \tilde{\mathbf{y}}_i^T \mathbf{b}_i - \tilde{\mathbf{r}}_i^k \tilde{\mathbf{y}}_i \right. \\ & \quad \left. - \rho \sum_{j \in \mathcal{N}_i} \left\| \tilde{\mathbf{y}}_i - \frac{\tilde{\mathbf{y}}_i^k + \tilde{\mathbf{y}}_j^k}{2} \right\|_2^2 \right\} \\ & \text{subject to} \quad \tilde{\mathbf{y}}_i \geq 0 \end{aligned} \quad (28)$$

with the update procedure for $\tilde{\mathbf{r}}_i$ given by

$$\tilde{\mathbf{r}}_i^{k+1} = \tilde{\mathbf{r}}_i^k + \rho \sum_{j \in \mathcal{N}_i} (\tilde{\mathbf{y}}_i^{k+1} - \tilde{\mathbf{y}}_j^{k+1}) \quad (29)$$

at iteration k .

We can further simplify the update procedures for the local dual variable in (28) by recognizing that the equality constraint in (4) corresponds to an unrestricted dual variable. Consequently, the update procedure in (28) simplifies to

$$\begin{aligned} & \underset{y_i, \lambda_i}{\text{maximize}} \quad \left\{ -f_i^*(y_i - \lambda_i \mathbf{1}_m) + \frac{1}{N} \mathbf{1}_m^T y_i - \lambda_i - \mathbf{r}_i^{kT} y_i \right. \\ & \quad \left. - \rho \sum_{j \in \mathcal{N}_i} \left\| y_i - \frac{y_i^k + y_j^k}{2} \right\|_2^2 \right\} \\ & \text{subject to} \quad y_i \geq 0 \end{aligned} \quad (30)$$

where $y_i \in \mathbb{R}^m$ denotes robot i 's dual variable for the first inequality constraint in (4) and $\lambda_i \in \mathbb{R}$ denotes its dual variable for the equality constraint, with the associated update procedure for $r_i \in \mathbb{R}^m$ given by

$$\mathbf{r}_i^{k+1} = \mathbf{r}_i^k + \rho \sum_{j \in \mathcal{N}_i} (y_i^{k+1} - y_j^{k+1}) \quad (31)$$

at iteration k .

Remark 1: In problems where the objective function $f_i(x_i)$ of robot i in (4) consists of a linear function, the Fenchel conjugate $f_i^*(y_i - \lambda_i \mathbf{1}_m)$ has a closed-form solution when

$$c_i - y_i + \lambda_i \mathbf{1}_m \geq 0 \quad (32)$$

with an optimal value of zero. In these problems, robot i updates y_i and λ_i as the solution of the optimization problem

$$\begin{aligned} & \underset{y_i, \lambda_i}{\text{maximize}} \quad \left\{ \frac{1}{N} \mathbf{1}_m^T y_i - \lambda_i - \mathbf{r}_i^{kT} y_i \right. \\ & \quad \left. - \rho \sum_{j \in \mathcal{N}_i} \left\| y_i - \frac{y_i^k + y_j^k}{2} \right\|_2^2 \right\} \\ & \text{subject to} \quad c_i - y_i + \lambda_i \mathbf{1}_m \geq 0 \\ & \quad y_i \geq 0 \end{aligned} \quad (33)$$

at iteration k .

The dual optimization problem of robot i in (30) does not involve the local constraints of other robots, noting that each robot only knows a subset of the problem constraints in (25). However, the equality constraints on the local dual variables in (25) ensure that the dual variable of each robot satisfies all the problem constraints.

Theorem 2 (Convergence of $\{\mathbf{y}^k\}$): The sequence of local dual variables $\{\mathbf{y}_i^k\}$ of robot i converges to the optimal dual solution $\mathbf{y}^*$, $\forall i \in \mathcal{V}$.

Proof: The proof follows along the same lines as the proof of Theorem 1. For completeness, we provide the proof in Appendix B. $\square$

Upon convergence of the dual variables, each robot computes its optimal primal solution from

$$\begin{aligned}
 & \underset{x_i}{\text{minimize}} \underset{y_i}{\text{maximize}} \left\{ f_i(x_i) + y_i^\top \left(-x_i + \frac{1}{N} \mathbf{1}_m \right) - r_i^{\star\top} y_i \right. \\
 & \quad \left. - \rho \sum_{j \in \mathcal{N}_i} \left\| y_i - \frac{\bar{y}_i + \bar{y}_j}{2} \right\|_2^2 \right\} \\
 & \text{subject to } y_i \geq 0 \\
 & \quad \mathbf{1}_m^\top x_i = 1 \\
 & \quad 0 \leq x_i \leq 1
 \end{aligned} \tag{34}$$

where $\bar{y}_i$ denotes the local dual variable of robot i computed at the last iteration of our distributed algorithm. The problem in (34) simplifies to

$$\begin{aligned}
 & \underset{x_i}{\text{minimize}} \underset{y_i}{\text{maximize}} \left\{ f_i(x_i) - \rho |\mathcal{N}_i| \|y_i - \nu_i^*(x_i)\|_2^2 \right. \\
 & \quad + \rho |\mathcal{N}_i| \|\nu_i^*(x_i)\|_2^2 \\
 & \quad \left. - \frac{\rho}{4} \sum_{j \in \mathcal{N}_i} \|\bar{y}_i + \bar{y}_j\|_2^2 \right\} \\
 & \text{subject to } y_i \geq 0 \\
 & \quad \mathbf{1}_m^\top x_i = 1 \\
 & \quad 0 \leq x_i \leq 1
 \end{aligned} \tag{35}$$

where

$$\nu_i^*(x_i) = \frac{1}{2\rho|\mathcal{N}_i|} \left(-x_i + \frac{1}{N} \mathbf{1}_m - r_i^* + \rho \sum_{j \in \mathcal{N}_i} (\bar{y}_i + \bar{y}_j) \right) \tag{36}$$

which results in a closed-form solution for y_i , given by

$$y_i^*(x_i) = \max(0, \nu_i^*(x_i)) \tag{37}$$

with the resulting minimization problem

$$\begin{aligned}
 & \underset{x_i}{\text{minimize}} \left\{ f_i(x_i) - \rho |\mathcal{N}_i| \|\min(0, \nu_i^*(x_i))\|_2^2 \right. \\
 & \quad \left. + \rho |\mathcal{N}_i| \|\nu_i^*(x_i)\|_2^2 \right\} \\
 & \text{subject to } \mathbf{1}_m^\top x_i = 1 \\
 & \quad 0 \leq x_i \leq 1
 \end{aligned} \tag{38}$$

for x_i . After further simplifying the problem in (38), robot i computes x_i from the optimization problem

$$\begin{aligned}
 & \underset{x_i}{\text{minimize}} \left\{ f_i(x_i) + \frac{\rho |\mathcal{N}_i|}{2} \|\nu_i^*(x_i)\|_2^2 \right. \\
 & \quad \left. + \frac{\rho |\mathcal{N}_i|}{2} |\nu_i^*(x_i)|^\top \nu_i^*(x_i) \right\} \\
 & \text{subject to } \mathbf{1}_m^\top x_i = 1 \\
 & \quad 0 \leq x_i \leq 1
 \end{aligned} \tag{39}$$

Algorithm 2: Distributed Multirobot Dual Algorithm for Task Assignment Problems (MURD-TAP).

Initialization:

$$y_i^0 \in \mathbb{R}^m, \lambda_i^0 \in \mathbb{R} \quad \forall i \in \mathcal{V}$$

$$r_i^0 \leftarrow 0 \quad \forall i \in \mathcal{V}$$

$$k \leftarrow 0$$

do in parallel $\forall i \in \mathcal{V}$

$$(y_i^{k+1}, \lambda_i^{k+1}) \leftarrow \text{Procedure (30)}$$

$$r_i^{k+1} \leftarrow \text{Procedure (31)}$$

$$k \leftarrow k + 1$$

while not converged or stopping criterion is not met;

Recovery:

$$x_i \leftarrow \text{Procedure (39)} \quad \forall i \in \mathcal{V}$$

after the last iteration of our distributed algorithm, as determined by a specified stopping criterion.

Algorithm 2 outlines our distributed dual algorithm for task assignment problems. Each robot does not maintain the slack variables γ and ζ and the Lagrange multipliers p and w . Furthermore, the updates in (30) only require each robot to communicate its dual variables with its neighbors, without communicating its task assignment, minimizing the amount of potentially sensitive data exchanged by each robot. MURD-TAP does not require a feasible initialization of the local dual variables of each robot.

Theorem 3: The primal variable of all robots $x^S = [x_1^\top, \dots, x_N^\top]^\top$, computed in (39), corresponds to the optimal solution x^* of (18).

Proof: We provide the proof in Appendix C. $\square$

C. Distributed Inexact Dual Algorithm

In the MUR-TAP and MURD-TAP algorithms, each robot solves the optimization problem arising in its primal update procedure through a nested iterative method to update its local variables. In problems where each robot has limited access to computational resources, solving these optimization problems may prove challenging for each robot. Consequently, we derive a distributed algorithm with simpler update procedures for task assignment problems. We express the problem in (4) as

$$\begin{aligned}
 & \underset{x}{\text{minimize}} \sum_{i=1}^N f_i(x_i) \\
 & \text{subject to } \sum_{i=1}^N \left(\frac{1}{N} \mathbf{1}_m - x_i \right) \leq 0 \\
 & \quad \sum_{i=1}^N \left(G_i x_i - \frac{1}{N} \mathbf{1}_N \right) = 0 \\
 & \quad 0 \leq x_i \leq 1 \quad \forall i \in \mathcal{V}
 \end{aligned} \tag{40}$$

where $G_i \in \mathbb{R}^{N \times m}$. We set the i th row of G_i to $\mathbf{1}_m^\top$ with all other entries set to zero.

We express the Lagrangian of (40) as

$$\begin{aligned} \mathcal{L}_p(x, y, \lambda) = & \sum_{i=1}^N f_i(x_i) + y^\top \sum_{i=1}^N \left(\frac{1}{N} \mathbf{1}_m - x_i \right) \\ & + \lambda^\top \sum_{i=1}^N \left(G_i x_i - \frac{1}{N} \mathbf{1}_N \right) \end{aligned} \quad (41)$$

with dual variables $y \in \mathbb{R}^m$ and $\lambda \in \mathbb{R}^N$ where $y \geq 0$, without relaxing the box constraints on x . From (41), we obtain the dual problem

$$\begin{aligned} & \underset{y, \lambda}{\text{maximize}} \sum_{i=1}^N \phi_i(y, \lambda) \\ & \text{subject to } y \geq 0 \end{aligned} \quad (42)$$

where

$$\begin{aligned} \phi_i(y, \lambda) = & \inf_{x_i} \left\{ f_i(x_i) + y^\top \left(\frac{1}{N} \mathbf{1}_m - x_i \right) \right. \\ & \left. + \lambda^\top \left(G_i x_i - \frac{1}{N} \mathbf{1}_N \right) \right\} \end{aligned} \quad (43)$$

with $0 \leq x_i \leq 1$. To derive a distributed method for solving (42), we assign local copies of the dual variables y and λ to each robot, with the resulting optimization problem given by

$$\begin{aligned} & \underset{y, \lambda}{\text{maximize}} \sum_{i=1}^N \phi_i(y_i, \lambda_i) \\ & \text{subject to } y_i \geq 0 \quad \forall i \in \mathcal{V} \\ & y_i = y_j, \lambda_i = \lambda_j \quad \forall j \in \mathcal{N}_i, \forall i \in \mathcal{V} \end{aligned} \quad (44)$$

where $y = [y_i^\top, \forall i \in \mathcal{V}]^\top$ and $\lambda = [\lambda_i^\top, \forall i \in \mathcal{V}]^\top$. Following a similar proof to that of Proposition 2, the optimization problems in (42) and (44) have the same optimal solution and optimal objective value.

Taking the same approach employed in Section VI-B, we derive a distributed algorithm for solving the optimization problem in (44). With this approach, robot i computes its local variables y_i and λ_i from

$$\begin{aligned} & \underset{y_i, \lambda_i}{\text{maximize}} \left\{ \phi_i(y_i, \lambda_i) - \eta_i^{k\top} y_i - \psi_i^{k\top} \lambda_i \right. \\ & \quad - \rho \sum_{j \in \mathcal{N}_i} \left\| y_i - \frac{y_i^k + y_j^k}{2} \right\|_2^2 \\ & \quad \left. - \rho \sum_{j \in \mathcal{N}_i} \left\| \lambda_i - \frac{\lambda_i^k + \lambda_j^k}{2} \right\|_2^2 \right\} \\ & \text{subject to } y_i \geq 0 \end{aligned} \quad (45)$$

where $\eta_i \in \mathbb{R}^m$ and $\psi_i \in \mathbb{R}^N$ denote Lagrange multipliers for the equality constraints in (44). The optimization problem in (45) represents a min-max optimization problem, which in general can be difficult to solve. However, we leverage the existence of

a saddle point for the Lagrangian of (40) to compute a solution for (45). We express the optimization problem in (45) as

$$\begin{aligned} & \underset{y_i, \lambda_i}{\text{maximize}} \underset{x_i}{\text{minimize}} \left\{ f_i(x_i) + y_i^\top \left(\frac{1}{N} \mathbf{1}_m - x_i \right) \right. \\ & \quad + \lambda_i^\top \left(G_i x_i - \frac{1}{N} \mathbf{1}_N \right) \\ & \quad - \eta_i^{k\top} y_i - \psi_i^{k\top} \lambda_i \\ & \quad - \rho \sum_{j \in \mathcal{N}_i} \left\| y_i - \frac{y_i^k + y_j^k}{2} \right\|_2^2 \\ & \quad \left. - \rho \sum_{j \in \mathcal{N}_i} \left\| \lambda_i - \frac{\lambda_i^k + \lambda_j^k}{2} \right\|_2^2 \right\} \\ & \text{subject to } 0 \leq x_i \leq 1 \\ & y_i \geq 0 \end{aligned} \quad (46)$$

which simplifies to

$$\begin{aligned} & \underset{y_i, \lambda_i}{\text{maximize}} \underset{x_i}{\text{minimize}} \left\{ f_i(x_i) - \rho |\mathcal{N}_i| \|y_i - \nu_i^k(x_i)\|_2^2 \right. \\ & \quad + \rho |\mathcal{N}_i| \|\nu_i^k(x_i)\|_2^2 \\ & \quad - \rho |\mathcal{N}_i| \|\lambda_i - \ell_i^k(x_i)\|_2^2 \\ & \quad \left. + \rho |\mathcal{N}_i| \|\ell_i^k(x_i)\|_2^2 + \theta_i^k \right\} \\ & \text{subject to } 0 \leq x_i \leq 1 \\ & y_i \geq 0 \end{aligned} \quad (47)$$

where

$$\begin{aligned} \nu_i^k(x_i) &= \frac{1}{2\rho|\mathcal{N}_i|} \left(\frac{1}{N} \mathbf{1}_m - x_i - \eta_i^k + \rho \sum_{j \in \mathcal{N}_i} (y_i^k + y_j^k) \right) \\ \ell_i^k(x_i) &= \frac{1}{2\rho|\mathcal{N}_i|} \left(G_i x_i - \frac{1}{N} \mathbf{1}_N - \psi_i^k + \rho \sum_{j \in \mathcal{N}_i} (\lambda_i^k + \lambda_j^k) \right) \end{aligned} \quad (48)$$

and θ_i^k are independent of the dual variable y_i . The existence of a saddle point enables us to swap the order of the optimization problems to solve (47), yielding a closed-form solution for y_i with

$$\begin{aligned} y_i^{k+1} &= \max(0, \nu_i^k(x_i^{k+1})) \\ \lambda_i^{k+1} &= \ell_i^k(x_i^{k+1}) \end{aligned} \quad (49)$$

where the max operator works elementwise [50]. By swapping the order of the optimization problems, the minimization problem for x_i simplifies to

$$\begin{aligned} & \underset{x_i}{\text{minimize}} \left\{ f_i(x_i) - \rho |\mathcal{N}_i| \|\min(0, \nu_i^k(x_i))\|_2^2 \right. \\ & \quad \left. + \rho |\mathcal{N}_i| \|\nu_i^k(x_i)\|_2^2 + \rho |\mathcal{N}_i| \|\ell_i^k(x_i)\|_2^2 \right\} \\ & \text{subject to } 0 \leq x_i \leq 1 \end{aligned} \quad (50)$$

which represents a nonlinear optimization problem. We note that the optimization problem in (50) might be challenging to solve in some situations. Hence, we utilize a proximal gradient update scheme for solving the problem in (50). We take a first-order approximation of the objective function of (50) at x_i^k and solve the resulting optimization problem

$$\begin{aligned} \underset{x_i}{\text{minimize}} \quad & \left\{ \nabla f_i(x_i^k)^\top x_i - \kappa_i(x_i^k)^\top x_i - \nu_i^k(x_i^k)^\top x_i \right. \\ & \left. + \ell_i^k(x_i^k)^\top G_i x_i + \frac{1}{2\beta_i} \|x_i - x_i^k\|_2^2 \right\} \end{aligned} \quad (51)$$

where

$$\kappa_i(x_i^k) = -\min(0, \nu_i^k(x_i^k)) \quad (52)$$

which yields the closed-form solution

$$\hat{x}_i^k = x_i^k - \beta_i (\nabla f_i(x_i^k) - \kappa_i(x_i^k) - \nu_i^k(x_i^k) + G_i^\top \ell_i^k(x_i^k)) \quad (53)$$

equivalent to the gradient descent update of x_i with step size β_i at iteration k . Subsequently, we take a proximal projection of the solution in (53) using the proximal operator $\rho_i(\hat{x}_i^k)$ defined as the solution to the optimization problem

$$\begin{aligned} \underset{x_i}{\text{minimize}} \quad & \|x_i - \hat{x}_i^k\|_2^2 \\ \text{subject to} \quad & 0 \leq x_i \leq 1 \end{aligned} \quad (54)$$

which admits a closed-form solution. Consequently, robot i updates x_i at iteration k using the closed-form procedure

$$x_i^{k+1} = \min(1, \max(0, \hat{x}_i^k)) \quad (55)$$

before computing y_i and λ_i using (49). In addition, robot i updates η_i and ψ_i using

$$\begin{aligned} \eta_i^{k+1} &= \eta_i^k + \rho \sum_{j \in N_i} (y_i^{k+1} - y_j^{k+1}) \\ \psi_i^{k+1} &= \psi_i^k + \rho \sum_{j \in N_i} (\lambda_i^{k+1} - \lambda_j^{k+1}) \end{aligned} \quad (56)$$

at iteration k .

We outline our distributed inexact dual algorithm for the multirobot task assignment problem in Algorithm 3. Note that our distributed algorithm does not require each robot to utilize a nested iterative method to update their local variables. Rather, all the robots update their local variables using closed-form solutions, which involve arithmetic operations.

Theorem 4 (Convergence of $(x^k, \mathbf{y}^k, \lambda^k)$): The iterates $(x_i^k, y_i^k, \lambda_i^k)$ of robot i converge to an optimal primal-dual solution pair $(x_i^*, y_i^*, \lambda_i^*)$ of (40), $\forall i \in \mathcal{V}$.

Proof: Refer to [51] for the proof. $\square$

D. Algorithm Selection

Our algorithms apply to a variety of task assignment problems, given by (4), including the classical task assignment problem with linear objective functions. The selection of the most efficient algorithm for a given multirobot task assignment problem depends on the relative availability of computation and communication resources at each robot and the privacy

Algorithm 3: Distributed Multirobot Inexact Dual Algorithm for Task Assignment Problems (MURID-TAP).

Initialization:

$$x_i^0 \in \mathbb{R}^m, y_i^0 \in \mathbb{R}^m, \lambda_i^0 \in \mathbb{R}^N \quad \forall i \in \mathcal{V}$$

$$\eta_i^0 \leftarrow 0, \psi_i^0 \leftarrow 0 \quad \forall i \in \mathcal{V}$$

$$k \leftarrow 0$$

do in parallel $\forall i \in \mathcal{V}$

$$x_i^{k+1} \leftarrow \text{Procedure (55)}$$

$$(y_i^{k+1}, \lambda_i^{k+1}) \leftarrow \text{Procedure (49)}$$

$$(\eta_i^{k+1}, \psi_i^{k+1}) \leftarrow \text{Procedure (56)}$$

$$k \leftarrow k + 1$$

while *not converged or stopping criterion is not met;*

requirements in the given problem. To guide the selection of an efficient algorithm, we examine the computational, storage, and communication requirements of each algorithm.

1) *Computational Complexity:* The MUR-TAP algorithm requires each robot to solve the constrained optimization problem in (15) to compute its task assignment. The optimization problem can be solved using an interior-point method, which involves factorizing the matrix associated with the Karush–Kuhn–Tucker (KKT) necessary conditions for optimality—the KKT matrix. Assuming that the KKT matrix is positive definite, factorizing the matrix can be performed through Cholesky decomposition at a cost of $O((Nm)^3)$ floating-point operations (FLOPS), where we have retained the dominant terms in quantifying the number of FLOPS required. In addition, robot i updates its local Lagrange multiplier q_i in the MUR-TAP algorithm using $O(Nm)$ FLOPS. Hence, the MUR-TAP algorithm has a net computational complexity of $O((Nm)^3)$ FLOPS at each iteration.

In our distributed exact dual algorithm, MURD-TAP, each robot updates its dual variables by solving the optimization problem in (30). The optimization problem can be solved using an interior-point method, which requires $O(m^3)$ FLOPS to factorize the associated KKT matrix. Furthermore, each robot updates its Lagrange multiplier using (31) with $O(m)$ FLOPS. Consequently, the MURD-TAP algorithm has a net computational complexity of $O(m^3)$ FLOPS at each iteration.

MURID-TAP does not require a nested iterative method for any optimization problem. Each robot updates its local variables using the closed-form procedures in (49), (55), and (56). As a result, robot i requires $O(m + N)$ FLOPS to update its local variables at each iteration.

2) *Communication Complexity:* In the MUR-TAP algorithm, each robot shares its local primal variable with its neighbors at each iteration. As such, each robot transmits $O(Nm)$ bits of information to its neighbors and, likewise, receives $O(Nm)$ bits of information. The MURD-TAP algorithm requires each robot to communicate its local dual variables to its neighbors. Hence, the MURD-TAP algorithm has a communication complexity of $O(m)$ bits. Similarly, the MURID-TAP algorithm requires each

TABLE I
COMPLEXITY OF THE MULTIROBOT TASK ASSIGNMENT ALGORITHMS PER ITERATION

Algorithm	Computation	Communication	Data Storage
MUR-TAP	$O((Nm)^3)$	$O(Nm)$	$2(Nm)Q$
MURD-TAP	$O(m^3)$	$O(m)$	$(2m+1)Q$
MURID-TAP	$O(m+N)$	$O(m+N)$	$(3m+2N)Q$

Computational and communication complexity is measured in FLOPS, while data storage complexity is measured in bits. We denote the number of bits required to represent each component of the optimization variable as Q .

robot to share its local dual variables with its neighbors, resulting in a communication complexity of $O(m+N)$ bits.

3) *Data Storage Complexity*: Each robot maintains a local primal variable and a local Lagrange multiplier for the equality constraints between its primal solution and that of its neighbors in the MUR-TAP algorithm. We assume that each robot represents its optimization variables using Q bits. When these variables are represented as double-precision floating-point numbers, $Q = 64$ bits. With this assumption, each robot requires $2NmQ$ bits for storing its local variables in the MUR-TAP algorithm. In the MURD-TAP algorithm, each robot maintains a local dual variable and a Lagrange multiplier, which require $(2m+1)Q$ bits for data storage. The primal solution can be recovered upon termination of the MURD-TAP algorithm. In contrast, the MURID-TAP algorithm requires each robot to maintain a local primal and dual variable and Lagrange multipliers for the equality constraints between its dual variable and that of its neighbors. As a result, in the MURID-TAP algorithm, robot i requires $(3m+2N)Q$ bits for storing its local variables.

4) *Selection Guide*: We summarize the results of the complexity analysis in Table I. The computational complexity analysis reveals that the per-iteration computational complexity of the MUR-TAP algorithm scales cubically the number of robots N and the number of tasks m in the task assignment problem in (4), while the per-iteration computational complexity of the MURD-TAP algorithm scales cubically in the number of tasks m only. In contrast, the MURID-TAP algorithm provides the most efficient per-iteration computational complexity, scaling linearly in the sum of the number of robots and tasks $(m+N)$. Consequently, the MURID-TAP algorithm provides an efficient method for solving multirobot problems when access to adequate computation resources poses a limiting constraint (and in problems with a large number of robots/tasks). However, when the availability of computation resources does not prove prohibitive, the MUR-TAP and MURD-TAP algorithms should be considered, as, generally, the MURID-TAP algorithm requires a greater number of iterations for convergence, given its inexact update procedures.

In problems with low communication bandwidth and situations where communication between robots comes at a premium, the communication complexity of the algorithms plays a critical role in selecting an efficient distributed algorithm. MURD-TAP provides the lowest communication complexity, independent of the number of robots in the problem; however, MURID-TAP also scales efficiently with respect to its communication complexity, scaling linearly in the sum of the number of robots and tasks.

In situations with limited local data storage resources, MURD-TAP and MURID-MP algorithms provide more efficient methods for solving multirobot problems. However, MURID-TAP scales linearly in the number of robots, which can be consequential in problems with a large number of robots. In contrast, the data storage complexity of MURD-TAP does not depend on the number of robots. In general, the selection of an efficient algorithm depends on the relative computation, communication, and data storage resources available to each robot in the multirobot problem.

VII. SIMULATIONS

In this section, we examine the performance of our distributed algorithms in the multirobot task assignment problem. We compare our methods to other distributed methods in each of these problems, assessing the convergence rates of each method to the optimal task assignment, computed centrally after collating all the problem data. We begin with the task assignment problem with linear objective functions and an equal number of robots and tasks, which is amenable to other distributed task assignment algorithms. Thereafter, we consider the multirobot task assignment problem with nonlinear objective functions and an unequal number of robots and tasks, a problem that is unsuitable for many distributed task assignment algorithms. Finally, we apply our algorithms to the multirobot persistent surveillance problem, where we consider periodic assignments of robots to aerial surveillance stations over time, with constraints on the minimum allowable capacity level of each robot's battery. We execute all the algorithms on a laptop computer with an Intel i7 processor with 16-GB RAM and use the interior-point methods for quadratic programming available in Gurobi [52].

A. Linear Multirobot Task Assignment Problem

We consider the multirobot task assignment problem in (2) where the objective function $f(x)$ consists of a sum of linear functions, given by

$$f(x) = \sum_{i=1}^N c_i^T x_i \quad (57)$$

with $x_i \in \mathbb{R}^m$ denoting the optimization variable of robot i and $x = [x_1^T, \dots, x_N^T]^T \in \mathbb{R}^{Nm}$. We examine the convergence rate of our algorithms—MUR-TAP (primal algorithm), MURD-TAP (exact dual algorithm), and MURID-TAP (inexact dual algorithm)—to the optimal task assignment, assessing the convergence of each algorithm in terms of the percentage relative error with respect to the optimal solution of the problem. We define the relative error with respect to the optimal task assignment as

$$\text{RE}(x) = \frac{\|x - x^*\|_2}{\|x^*\|_2} \quad (58)$$

where x^* denotes the optimal task assignment. In each task assignment problem, we randomly generate the vectors in the objective function.

1) *Convergence to the Optimal Solution*: We examine the convergence rate of each algorithm to the optimal solution on randomly generated connected communication networks. We

TABLE II

NUMBER OF OPTIMIZATION VARIABLES (# OF VAR.) AND THE MEAN AND STANDARD DEVIATION OF THE NUMBER OF COMMUNICATION ROUNDS (# OF COMM.) AND TOTAL COMPUTATION TIME (COMP. TIME) IN MILLISECONDS, PER ROBOT, IN 60 TASK ASSIGNMENT PROBLEMS WITH FIVE ROBOTS AND FIVE TASKS

Algorithm	# of Var.	# of Comm.	Comp. Time (ms)
MUR-TAP	25	25 ± 14	4.164 ± 2.306
MURD-TAP	6	31 ± 17	3.730 ± 1.934
MURID-TAP	15	45 ± 20	1.608 ± 0.641

begin with a task assignment problem with five robots and five tasks. In Table II, we provide the number of optimization variables maintained by each robot and the mean and standard deviation of the number of communication rounds and total computation time in milliseconds required by each robot in these algorithms to achieve a relative error of at most $1e^{-11}\%$ across 60 problems.

From Table II, each robot in MURD-TAP optimizes over six variables, while each robot in MUR-TAP and MURID-TAP optimizes over 25 variables and 15 variables, respectively. Consequently, the MURD-TAP algorithm requires the least data storage capacity, which could be essential in robots with limited onboard storage. Although each robot in MUR-TAP optimizes over the greatest number of optimization variables compared to the other algorithms, each robot requires the fewest number of communication rounds to compute the optimal task assignment. In MUR-TAP, each robot computes the optimal task assignment within a mean of 25 communication rounds, while each robot in MURD-TAP and MURID-TAP requires a mean of 31 and 45 communication rounds, respectively. However, the relatively larger size of the optimization problem contributes to the longer computation time required by MUR-TAP for convergence, with each robot obtaining its optimal assignment after a mean computation time of 4.164 ms. With MURID-TAP, each robot takes the shortest mean computation time of about 1.608 ms to obtain the optimal task assignment; however, MURID-TAP requires the greatest number of communication rounds for convergence to the optimal task assignment. MURD-TAP requires a shorter mean computation time of 3.730 ms to converge to the optimal solution compared to MUR-TAP, but the convergence of MURD-TAP is attained after a greater number of communication rounds.

In Fig. 1, we show the relative error of the local variables of all robots with the MUR-TAP, MURD-TAP, and MURID-TAP algorithms during one trial. MUR-TAP attains the fastest convergence rate, with each robot computing an optimal task assignment within 35 communication rounds. In contrast, with MURD-TAP, each robot requires about 40 communication rounds to compute its optimal task assignment. MURID-TAP achieves the slowest convergence rate among the three algorithms, requiring about 50 communication rounds for convergence.

Next, we examine the multirobot task assignment problem with ten robots and ten tasks. Table III provides the number of optimization variables maintained by each robot and the mean

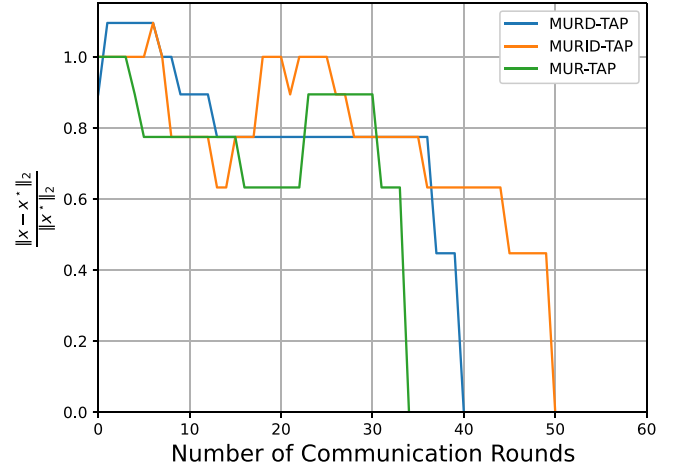


Fig. 1. Relative error of MUR-TAP, MURD-TAP, and MURID-TAP on the multirobot task assignment problem with five robots and five tasks. MUR-TAP provides the fastest convergence rate, converging within 35 communication rounds. With MURD-TAP, the relative error of the local variables of all robots converges to zero faster than that of MURID-TAP.

TABLE III

NUMBER OF OPTIMIZATION VARIABLES (# OF VAR.) AND THE MEAN AND STANDARD DEVIATION OF THE NUMBER OF COMMUNICATION ROUNDS (# OF COMM.) AND CUMULATIVE COMPUTATION TIME (COMP. TIME) IN MILLISECONDS, PER ROBOT, IN 60 TASK ASSIGNMENT PROBLEMS WITH TEN ROBOTS AND TEN TASKS

Algorithm	# of Var.	# of Comm.	Comp. Time (ms)
MUR-TAP	100	85 ± 38	29.421 ± 12.832
MURD-TAP	11	39 ± 22	5.68 ± 3.055
MURID-TAP	30	87 ± 38	1.526 ± 0.603

and standard deviation of the number of communication rounds and cumulative computation time in milliseconds required by each robot to achieve a relative error of at most $1e^{-11}\%$ across 60 problems.

As in the problem with five robots, MUR-TAP requires the greatest number of optimization variables, requiring each robot to maintain 100 variables, which results in a notable increase in the mean computation time per robot. In MUR-TAP, each robot takes a mean computation time of over 29 ms. In contrast, each robot in MURD-TAP maintains 11 variables, the smallest number of optimization variables maintained by each robot across all the algorithms. In addition, MURD-TAP requires the fewest number of communication rounds for convergence, with each robot computing the optimal task assignment within a mean of 39 communication rounds. Although each robot in MURID-TAP requires the greatest number of communication rounds to compute the optimal solution, MURID-TAP offers the shortest mean computation time of about 1.526 ms, compared to a mean computation time of 5.68 ms required by MURD-TAP. In MURID-TAP, each robot does not utilize a nested iterative method to update its local variables; rather, each robot updates its local variables using simple closed-form solutions, contributing to the shorter cumulative computation time required by MURID-TAP.

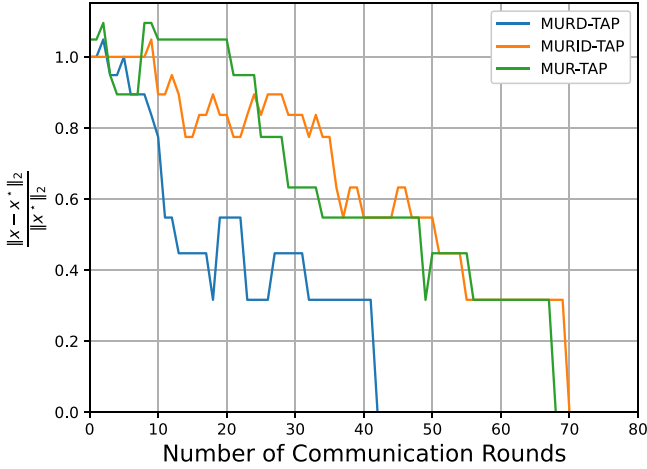


Fig. 2. Relative error of MUR-TAP, MURD-TAP, and MURID-TAP on the multirobot task assignment problem with ten robots and ten tasks. Compared to the task assignment problem with five robots, each robot in all the algorithms requires a greater number of communication rounds to compute its optimal task assignment. In this trial, MURD-TAP provides the fastest convergence rate compared to the other algorithms.

Fig. 2 shows the relative error of the task assignment of all robots in MUR-TAP, MURD-TAP, and MURID-TAP to an optimal task assignment in the problem with ten robots and ten tasks during one trial. In all the algorithms, each robot requires a greater number of communication rounds to compute its optimal task assignment compared to the number of communication rounds required in the task assignment problem with five robots and five tasks. In this trial, MUR-TAP achieves a slightly faster convergence rate compared to MURID-TAP, with each robot in MUR-TAP computing its optimal task assignment within 69 communication rounds, while MURID-TAP requires about 70 communication rounds for convergence. In MURD-TAP, each robot computes its optimal task assignment within 45 communication rounds.

In general, in problems with a relatively small number of robots and tasks, the MUR-TAP algorithm provides the fastest convergence rate compared to the other algorithms, given that each robot in the MUR-TAP algorithm computes a feasible assignment for all robots at each iteration of the algorithm. Each robot communicates with its neighbors across multiple iterations to reach consensus on a joint optimal assignment for all robots. In contrast, MURD-TAP and MURID-TAP do not provide a feasible assignment for all robots at each iteration until convergence. However, in larger problems, the effects of the computation and communication complexity of each algorithm become more apparent, resulting in slower convergence of the MUR-TAP algorithm.

In the subsequent simulations, we consider communication networks with different topologies in evaluating the convergence rate of our algorithm and, in addition, compare our algorithm to other distributed task assignment algorithms.

2) *Convergence Across Networks With Different Topologies:* In this simulation study, we assume that each robot has limited onboard storage, and thus, each robot requires a distributed algorithm with a low data storage overhead. Considering these

TABLE IV
MEAN AND STANDARD DEVIATION OF THE NUMBER OF COMMUNICATION ROUNDS (# OF COMM.) AND CUMULATIVE COMPUTATION TIME (COMP. TIME) IN MILLISECONDS OF MURD-TAP, PER ROBOT, IN 60 TASK ASSIGNMENT PROBLEMS WITH 20 ROBOTS AND 20 TASKS ON RANDOMLY GENERATED CONNECTED NETWORKS WITH DIFFERENT CONNECTIVITY RATIOS

Connectivity Ratio (κ)	# of Comm.	Comp. Time (ms)
0.253	94 $\pm$ 32	19.745 $\pm$ 6.819
0.595	61 $\pm$ 25	13.096 $\pm$ 5.166
0.879	56 $\pm$ 20	11.679 $\pm$ 4.044
1.000	54 $\pm$ 18	11.541 $\pm$ 3.790

limitations, we select the MURD-TAP algorithm to solve the task assignment problems in these scenarios. We examine the convergence rate of MURD-TAP to the optimal solution of the multirobot task assignment problem with 20 robots and 20 tasks across randomly generated connected communication networks with different connectivity ratios. We denote the connectivity ratio of a graph as $\kappa = \frac{2|E|}{N(N-1)}$, noting that a fully connected network has a connectivity ratio of 1. In Table IV, we show the mean and standard deviation of the number of communication rounds and cumulative computation time in milliseconds required by each robot to compute a task assignment with a relative error of at most $1e^{-11}\%$ across 60 problems.

From Table IV, we note that the number of communication rounds required for convergence decreases significantly as the connectivity ratio of a communication network increases from about 0.2 to 0.6, with a much smaller difference as the connectivity ratio increases beyond 0.6. Likewise, the cumulative computation time per robot decreases by about 7 ms over this range of the connectivity ratio. On fully connected communication networks, each robot requires about 54 communication rounds and about 11.541 ms in total computation time, to compute its optimal task assignment, compared to about 94 communication rounds and about 19.745 ms on communication networks with a connectivity ratio of 0.253. Consequently, the connectivity ratio of the underlying communication network influences the convergence rate of our algorithms, with a greater connectivity ratio corresponding to faster convergence.

In Fig. 3, we show the relative error of our algorithm during one trial across the communication networks presented in Table IV. Our algorithm attains its slowest convergence rate when $\kappa = 0.253$, the minimum connectivity ratio in Table IV, with a notable improvement in the convergence rate for larger values of κ . As a result, on communication networks with low connectivity ratios, each robot requires a greater number of communication rounds to achieve consensus with its neighbors on an optimal task assignment.

3) *Comparison to Benchmark Methods:* We assume that each robot has limited access to computational and data storage resources. Consequently, we examine the convergence rate of MURD-TAP and MURID-TAP for the multirobot task assignment problem in comparison to that of other distributed task assignment algorithms, including the consensus-based auction method (CBAA) [29], market-based consensus (MBC) method [17], and the distributed Hungarian (DH) method in [1].

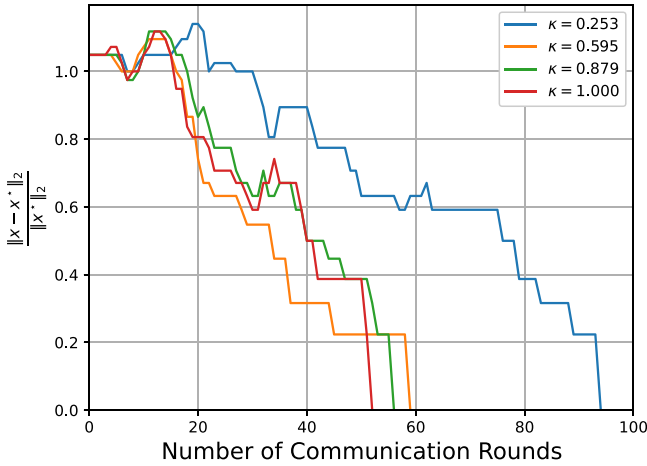


Fig. 3. Relative error of MURD-TAP on the multirobot task assignment problem with 20 robots and 20 tasks across randomly generated connected networks with different connectivity ratios. Our algorithm attains faster convergence rates in communication networks with larger connectivity ratios.

TABLE V

MEAN AND STANDARD DEVIATION OF THE NUMBER OF COMMUNICATION ROUNDS (# OF COMM.) AND CUMULATIVE COMPUTATION TIME (COMP. TIME) IN MILLISECONDS, PER ROBOT, OF DIFFERENT ALGORITHMS, AS WELL AS GUARANTEES ON PRODUCING AN OPTIMAL SOLUTION (OPT) AND AMENABILITY TO PROBLEMS WITH NONLINEAR OBJECTIVE FUNCTIONS (NL) IN 35 TASK ASSIGNMENT PROBLEMS WITH 50 ROBOTS AND 50 TASKS

Algorithm	# of Comm.	Comp. Time (ms)	Opt-NL
CBA [29]	5 ± 1	0.5 ± 0.271	N-N
MBC [17]	1049 ± 256	21.6 ± 7.7	Y-N
DH [1]	68 ± 9	210 ± 43.8	Y-N
MURD-TAP (ours)	172 ± 46	63.329 ± 16.591	Y-Y
MURID-TAP (ours)	385 ± 94	8.598 ± 1.846	Y-Y

In the Opt-NL column, “N” indicates false, while “Y” indicates true.

While the MBC, DH, MURD-TAP, and MURID-TAP algorithms provide an optimal task assignment, CBA does not guarantee an optimal task assignment. In addition, the MBC method requires all robots with conflicting task preferences to communicate with a merchant, which can be played by any of the robots, to resolve conflicting choices between the robots—which might be infeasible in randomly generated connected networks and chain networks, making the algorithm unsuitable in these problems. As a result, we consider a multirobot task assignment problem with 50 robots, communicating over a fully connected network, and 50 tasks. Table V presents the mean and standard deviation of the number of communication rounds and cumulative computation time in milliseconds required by each robot to compute a solution with a percentage relative error of at most $1e^{-11}\%$ with respect to an optimal task assignment across 35 problems. In addition, we indicate any guarantees provided by each algorithm on producing an optimal task assignment and the amenability of each algorithm to task assignment problems with nonlinear objective functions.

CBA requires the minimum cumulative computation time and the fewest number of communication rounds to compute a feasible task assignment for all robots. However, we note that CBA did not produce an optimal task assignment in

all the problems that we considered. Among the optimal task assignment algorithms—MBC, DH, MURD-TAP, and MURID-TAP—the MURID-TAP algorithm requires the shortest mean cumulative computation time to produce an optimal task assignment. However, each robot in the MURID-TAP algorithm requires a greater number of communication rounds to compute an optimal task assignment compared to the DH and MURD-TAP algorithms. Nonetheless, MURID-TAP requires fewer communication rounds to compute the optimal task assignment compared to the MBC algorithm, which requires the greatest number of communication rounds. In addition, the MBC algorithm is unsuitable for problems with nonlinear objective functions. Although the DH algorithm requires the fewest communication rounds among the optimal algorithms, the DH algorithm requires a greater cumulative computation time per robot compared to MURD-TAP and MURID-TAP. Moreover, the DH algorithm is not amenable to task assignment problems with nonlinear objective functions and general affine constraints. In contrast, MURD-TAP and MURID-TAP offer a balanced tradeoff with respect to computation time, the number of communication rounds required for convergence, and versatility, producing an optimal solution in general task assignment problems with nonlinear objective functions and affine constraints. MURD-TAP produces an optimal task assignment within fewer communication rounds while requiring a shorter cumulative computation time. Conversely, MURID-TAP achieves the fastest cumulative computation time with a greater number of communication rounds.

B. Nonlinear Multirobot Task Assignment Problem

We consider the multirobot task assignment problem in (2) with nonlinear objective functions and an unequal number of robots and tasks, where the objective function $f_i(x_i)$ of robot i is given by

$$f_i(x_i) = \sum_{\tau=1}^m \frac{1}{1 + \alpha_{i,\tau} e^{\beta_{i,\tau} x_{i,\tau}}} \quad (59)$$

with $x_i \in \mathbb{R}^m$ denoting the optimization variable of robot i . We examine the convergence rate of MURID-TAP on the nonlinear task assignment problem on randomly generated connected communication networks, noting the simple update procedures involved in the algorithm. We randomly generate the vectors $\alpha_i \in \mathbb{R}^m$ and $\beta_i \in \mathbb{R}^m$ arising in the objective function of robot i , with $\alpha_i \geq 0$ and $\beta_i \geq 0$. In general, the optimal solution of (2) does not always correspond to an integer-valued solution. However, when an integer-valued optimal solution exists, this solution corresponds to the optimal task assignment. We generate the objective function, given by (59), such that an integer-valued optimal solution exists.

For the nonlinear task assignment problem with $N = 20$ robots and $m = 15$ tasks, we examine the number of communication rounds and the cumulative computation time in milliseconds required by each robot to compute a solution with a relative error of at most $1e^{-11}\%$, with the mean and standard deviation across 60 problems provided in Table VI.

TABLE VI
MEAN AND STANDARD DEVIATION OF THE NUMBER OF COMMUNICATION
ROUNDS (# OF COMM.) AND CUMULATIVE COMPUTATION TIME (COMP. TIME)
IN MILLISECONDS, PER ROBOT, IN 60 NONLINEAR TASK ASSIGNMENT
PROBLEMS WITH 20 ROBOTS AND 15 TASKS

Algorithm	# of Comm.	Comp. Time (ms)
MURID-TAP	276 ± 12	3.799 ± 0.205

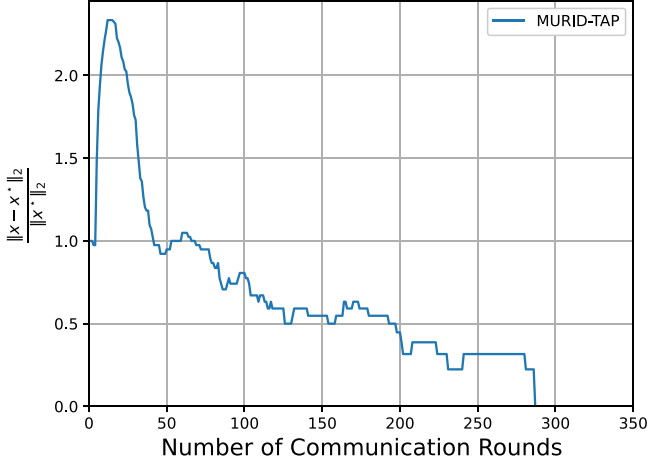


Fig. 4. Relative error of MURID-TAP on the nonlinear multirobot task assignment problem with 20 robots and 15 tasks on randomly generated connected networks.

With MURID-TAP, each robot computes its optimal task assignment efficiently, within a mean computation time of 3.799 ms. The cumulative computation required by each robot depends on the computational difficulty in evaluating the gradient of the objective function, required in the update procedure in (53). Fig. 4 shows the relative error of the solution computed by the robots during one trial. Each robot computes its optimal task assignment within 300 communication rounds.

C. Case Study: Persistent Surveillance

In the previous discussion, we considered single or one-shot task assignment problems; however, many multirobot systems require periodic assignments of robots to tasks over time, such as persistent surveillance problems with battery constraints. In this case study, we consider multirobot persistent surveillance where a group of robots must maintain coverage over a specified region. Given the battery constraints of each robot, the robots must alternate between performing the surveillance task and charging their batteries. As a result, the assignment of robots to the tasks must occur periodically. Moreover, robots at surveillance stations can only move to a charging station if another robot chooses to swap stations with it. We consider a problem with 12 robots and four dynamic surveillance stations, where the location of the surveillance stations is prescribed by a given trajectory. Fig. 5 shows four robots with spotlights surveilling a city, with eight charging stations (the green squares) located around the perimeter of the city. At this time instant, one robot (the robot within the red circle) is in the process of swapping

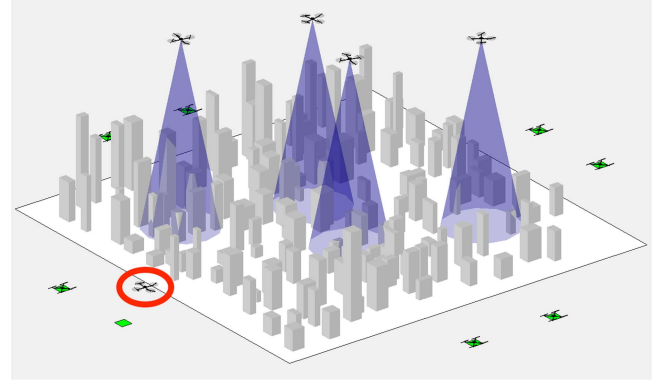


Fig. 5. One time step of a simulation. The charging stations are green squares around the perimeter of the city. The four drones over the city with spotlights are performing the surveillance task. One drone near the lower left (the robot within the red circle) is in the process of a replacement.

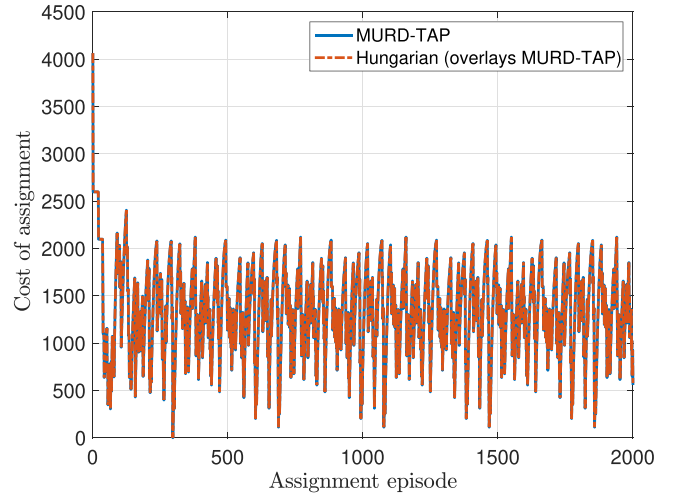


Fig. 6. Assignment of 12 robots to two surveillance stations on a persistent surveillance task using MURD-TAP. MURD-TAP produces the optimal task assignment at each episode, with a limit cycle emerging as the task proceeds.

stations, moving from a surveillance station to a charging station, as depicted in the Fig. 5. We use the terms *robot* and *drone* interchangeably. In the subsequent discussion, we provide the state dynamics models of the robots, their battery levels, and the surveillance stations. In addition, we specify the objective cost associated with each task assignment and compute the optimal task assignment for the surveillance problem.

1) *State Dynamics*: The state consists of a position $p_i \in \mathbb{R}^3$ and battery level $b_i \in \mathbb{Z}$ for each robot. At each time step, each robot can be at a charging station, at a surveillance station, or flying between stations. When at a charging station, the robot charges at a rate of r_c per time step up to a maximum battery level of $b_{\max}$. Otherwise, the robot discharges at a rate of r_d per time step down to a minimum battery level of 0, at which point the robot shutdowns as its battery capacity is fully drained. We provide the dynamics model for the battery capacity at time t in

$$b_i^{t+1} = \begin{cases} 0, & \text{if } b_i^t = 0 \\ \min(b_i^t + r_c, b_{\max}), & \text{if } i \text{ charging} \\ \max(b_i^t - r_d, 0), & \text{otherwise} \end{cases} \quad (60)$$

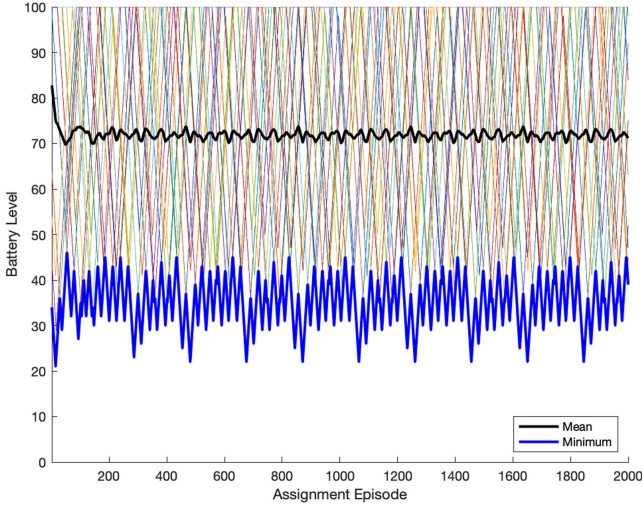


Fig. 7. Battery levels of the 12 robots during the simulation. The mean and minimum battery levels are highlighted to demonstrate the limit cycle and show that no batteries are completely discharged.

Stations follow known paths and are represented by functions $s_j(t) : \mathbb{Z} \rightarrow \mathbb{R}^3$. Charging stations are at fixed points in space, meaning that $s_j(t_1) = s_j(t_2) \forall t_1, t_2 \in \mathbb{Z}$. Surveillance stations may be at fixed points or follow paths.

Actions $a_i^t \in \mathbb{Z}$ dictate where each robot should go at time step t . If $a_i^t = j$, then robot i should stay at station j if it is there already or fly to station j if it is at a different station. If robot i needs to change its station, it chooses its goal location g_i^t according to

$$g_i^t = s_{a_i^t} \left(\min_{\Delta t \geq 1} \left(\Delta t \mid \|s_{a_i^t}(t + \Delta t) - p_i^t\|_2 \leq v\Delta t \right) \right) \quad (61)$$

where $v \in \mathbb{R}$ denotes its speed. The goal point is the first point along the assigned station's path that the robot can reach no later than it needs to in order to intercept the station when moving at a speed v . Once it has a goal location, the robot moves toward the goal point at a rate of v per time step, with the dynamics of the robot given by

$$p_i^{t+1} = p_i^t + \frac{g_i^t - p_i^t}{\|g_i^t - p_i^t\|_2} \min(\|g_i^t - p_i^t\|_2, v). \quad (62)$$

The surveillance paths are set so that they do not require movement faster than v .

2) *Costs and Assignments:* Assignments are influenced by constraints on the process of swapping between stations, as well as the costs associated with swapping stations. We delineate the problem constraints: First, each surveillance station must always have a drone at it. The drone at a surveillance station may only leave once a replacement has arrived, at which point it travels to the charging station that the replacement vacated. Second, once a replacement is sent, a second replacement may not be sent until the original replacement arrives. Third, a charging drone should not replace a surveillance drone if the latter has a higher battery level. Fourth, drones may not change assignments when traveling between stations. Fifth, drones may not swap between charging stations.

Costs determine the replacements. If drone i is charging at station j , the cost of staying at station j is

$$c_{ij} = -(b_i^t - b_{\max})^2 \quad (63)$$

which gives the drone less incentive to keep charging as its battery level rises. If, however, station j is a surveillance station, the cost of assigning drone i to be the replacement is

$$c_{ij} = d_{ij}^2 - (b_i^t - b_j^t) \quad (64)$$

where d_{ij} is the required travel distance, computed with (61), and b_j^t denotes the battery level of the drone currently at station j . This cost encourages replacements where the drone at the charging station has a much larger battery capacity and discourages replacements that require a large travel distance.

We compute the task assignments using MURID-TAP at each assignment episode, comparing the resulting assignments to the optimal solution obtained using the Hungarian method. In all episodes, MURID-TAP produces the optimal task assignment, depicted in Fig. 6 for 2000 assignment episodes. The assignment cost obtained using MURID-TAP overlays the assignment cost achieved using the Hungarian method at all episodes. A limit cycle of the total objective cost emerges as the task proceeds, with the robots alternating between surveillance and charging stations. Furthermore, we show the battery capacities of the 12 robots in Fig. 7, with a limit cycle also emerging. The battery capacity of each robot ranges from 0 (fully discharged) to 100 (fully charged). From Fig. 7, we note that the battery capacities of all robots remain above 20 at all episodes during the task, highlighting that no robot ends up with a fully discharged battery. In addition, the robots maintain a mean battery capacity of about 70 (see the supplementary video for more details).

VIII. CONCLUSION

In this article, we derived distributed algorithms for solving multirobot task assignment problems that only require a connected communication graph among the robots. We presented three related algorithms, all derived from variants of the ADMM, each with its own advantages and disadvantages. All three algorithms were proven to produce the optimal task assignment. The two algorithms based on the dual problem formulation also provided improved privacy to each robot, as each robot did not share its own task assignment while communicating with its neighbors. We provided extensive empirical results showing the tradeoffs among the three algorithms in terms of computational speed, storage requirements, and communication complexity. We also showed faster convergence rates attained by our algorithms in comparison to other distributed methods for multirobot task assignment. Finally, we demonstrated the potential of our algorithms in a more complex sequential task assignment problem in which a group of drones must coordinate with one another to perform aerial surveillance over a city while periodically returning to charging stations to charge their batteries. Our task assignment algorithms allowed the robots to achieve a limit cycle persistent monitoring behavior, balancing recharging and surveillance activities among themselves in a fully distributed fashion in perpetuity.

APPENDIX A PROOF OF THEOREM 1

We can express the primal mathematical program in (10) as

$$\begin{aligned} & \underset{\mathbf{x}, \boldsymbol{\sigma}}{\text{minimize}} \quad \sum_{i=1}^N f_i(\mathbf{x}_i) \\ & \text{subject to} \quad A_i \mathbf{x}_i \leq b_i \quad \forall i \in \mathcal{V} \\ & \quad \quad \mathcal{F}\mathbf{x} = \mathcal{G}\boldsymbol{\sigma} \end{aligned} \quad (65)$$

where $\mathcal{F} \in \mathbb{R}^{2|\mathcal{E}|Nm \times mN^2}$ and $\mathcal{G} \in \mathbb{R}^{2|\mathcal{E}|Nm \times 2|\mathcal{E}|Nm}$ define the mapping between the local $\mathbf{x}$ variable of each robot and the slack variables in $\boldsymbol{\sigma}$, as expressed by the equality constraints in (10). The augmented Lagrangian for the problem in (65) is given by

$$\begin{aligned} \mathcal{L}_a^{\mathcal{F}}(\mathbf{x}, \boldsymbol{\sigma}, \mu) &= \sum_{i=1}^N f_i(\mathbf{x}_i) + \mu^\top (\mathcal{F}\mathbf{x} - \mathcal{G}\boldsymbol{\sigma}) \\ & \quad + \frac{\rho}{2} \|\mathcal{F}\mathbf{x} - \mathcal{G}\boldsymbol{\sigma}\|_2^2 \end{aligned} \quad (66)$$

where $\mu \in \mathbb{R}^{2|\mathcal{E}|Nm}$ represents a Lagrange multiplier for the equality constraint in (65). We have not relaxed the inequality constraint in (65).

By simplifying the minimization problem for the primal update procedure in (15), robot i computes its primal variables from the problem

$$\begin{aligned} & \underset{\mathbf{x}_i}{\text{minimize}} \quad \left\{ f_i(\mathbf{x}_i) + \rho |\mathcal{N}_i| \|\mathbf{x}_i\|_2^2 + \mathbf{x}_i^\top \mathbf{q}_i^k \right. \\ & \quad \left. - \rho \mathbf{x}_i^\top \sum_{j \in \mathcal{N}_i} (\mathbf{x}_i^k + \mathbf{x}_j^k) \right\} \\ & \text{subject to} \quad A_i \mathbf{x}_i \leq b_i \end{aligned} \quad (67)$$

at each iteration. The local objective function of each robot as defined in (7) is convex, proper, and closed. Consequently, the minimization problem in (67) has a proper, closed, and coercive objective function. From the Weierstrass theorem [53], the minimization problem has a minimum value, which is attained at a solution $\tilde{\mathbf{x}}$, within the feasible set of the problem. As such, a minimizer exists for the minimization problems arising at each iteration of our algorithm.

At iteration k , the iterate $\mathbf{x}^{k+1}$ minimizes $\mathcal{L}_a^{\mathcal{F}}(\mathbf{x}, \boldsymbol{\sigma}^k, \mu^k)$. Likewise, $\boldsymbol{\sigma}^{k+1}$ minimizes $\mathcal{L}_a^{\mathcal{F}}(\mathbf{x}^{k+1}, \boldsymbol{\sigma}, \mu^k)$. From the optimality conditions arising in the primal update procedure, the iterates generated by our algorithm satisfy

$$\begin{aligned} & \frac{1}{\rho} \left(\|\mu^k - \mu^*\|_2^2 - \|\mu^{k+1} - \mu^*\|_2^2 \right) \\ & + \rho \left(\|\mathcal{G}(\boldsymbol{\sigma}^k - \boldsymbol{\sigma}^*)\|_2^2 - \|\mathcal{G}(\boldsymbol{\sigma}^{k+1} - \boldsymbol{\sigma}^*)\|_2^2 \right) \\ & \geq \rho \|\mathcal{F}\mathbf{x}^{k+1} - \mathcal{G}\boldsymbol{\sigma}^{k+1}\|_2^2 + \rho \|\mathcal{G}(\boldsymbol{\sigma}^{k+1} - \boldsymbol{\sigma}^k)\|_2^2 \end{aligned} \quad (68)$$

at each iteration, where $(\mathbf{x}^*, \boldsymbol{\sigma}^*, \mu^*)$ represents a saddle point of the Lagrangian of (65). From (68), the weighted error between

the iterates $(\boldsymbol{\sigma}^k, \mu^k)$ and $(\boldsymbol{\sigma}^*, \mu^*)$

$$\frac{1}{\rho} \|\mu^k - \mu^*\|_2^2 + \rho \|\mathcal{G}(\boldsymbol{\sigma}^k - \boldsymbol{\sigma}^*)\|_2^2 \quad (69)$$

decreases sufficiently at each iteration, which shows that the iterates $(\mathcal{G}\boldsymbol{\sigma}^k, \mu^k)$ are bounded. Summing (68) over all iterations results in

$$\begin{aligned} & \rho \sum_{k=0}^{\infty} \left(\|\mathcal{F}\mathbf{x}^{k+1} - \mathcal{G}\boldsymbol{\sigma}^{k+1}\|_2^2 + \|\mathcal{G}(\boldsymbol{\sigma}^{k+1} - \boldsymbol{\sigma}^k)\|_2^2 \right) \\ & \leq \frac{1}{\rho} \|\mu^0 - \mu^*\|_2^2 + \rho \|\mathcal{G}(\boldsymbol{\sigma}^0 - \boldsymbol{\sigma}^*)\|_2^2 \end{aligned} \quad (70)$$

which shows that

$$\begin{aligned} \mathcal{F}\mathbf{x}^{k+1} - \mathcal{G}\boldsymbol{\sigma}^{k+1} &\rightarrow 0 \quad \text{and} \\ \mathcal{G}(\boldsymbol{\sigma}^{k+1} - \boldsymbol{\sigma}^k) &\rightarrow 0 \end{aligned} \quad (71)$$

as $k \rightarrow \infty$. Consequently, the local primal variables of all robots converge to the same solution, satisfying the equality constraints in (10). In addition, the iterates satisfy

$$\begin{aligned} & \sum_{i=1}^N f_i(\mathbf{x}_i^{k+1}) - f_i(\mathbf{x}_i^*) \\ & \leq -(\mathcal{F}\mathbf{x}^{k+1} - \mathcal{G}\boldsymbol{\sigma}^{k+1})^\top \mu^{k+1} \\ & \quad + \rho (\mathcal{F}\mathbf{x}^{k+1} - \mathcal{G}\boldsymbol{\sigma}^{k+1})^\top \mathcal{G}(\boldsymbol{\sigma}^{k+1} - \boldsymbol{\sigma}^k) \\ & \quad - \rho (\mathcal{G}(\boldsymbol{\sigma}^{k+1} - \boldsymbol{\sigma}^*))^\top \mathcal{G}(\boldsymbol{\sigma}^{k+1} - \boldsymbol{\sigma}^k) \end{aligned} \quad (72)$$

along with

$$\sum_{i=1}^N f_i(\mathbf{x}_i^{k+1}) - f_i(\mathbf{x}_i^*) \geq -(\mathcal{F}\mathbf{x}^{k+1} - \mathcal{G}\boldsymbol{\sigma}^{k+1})^\top \mu^{k+1} \quad (73)$$

at each iteration, where $\mathbf{x}_i^* = \mathbf{x}^*$. As $k \rightarrow \infty$,

$$\sum_{i=1}^N f_i(\mathbf{x}_i^{k+1}) - f_i(\mathbf{x}_i^*) \rightarrow 0 \quad (74)$$

from the boundedness of $\mathcal{G}(\boldsymbol{\sigma}^{k+1} - \boldsymbol{\sigma}^*)$ and (71), showing that the objective value converges to its optimal value. Furthermore, the iterates $(\mathbf{x}^k, \boldsymbol{\sigma}^k, \mu^k)$ converge to $(\mathbf{x}^*, \boldsymbol{\sigma}^*, \mu^*)$, the saddle point of the Lagrangian. Refer to [54] and [55] for a detailed proof.

APPENDIX B PROOF OF THEOREM 2

The proof follows along the same lines as the proof of Theorem 1. For completeness, we provide the proof here. We can express the mathematical program in (25) as

$$\begin{aligned} & \underset{\mathbf{y}, \boldsymbol{\vartheta}}{\text{maximize}} \quad \sum_{i=1}^N (-f_i^*(-A_i^\top y_i) - y_i^\top b_i) \\ & \text{subject to} \quad y_i \geq 0 \quad \forall i \in \mathcal{V} \\ & \quad \quad \tilde{\mathcal{F}}\mathbf{y} = \tilde{\mathcal{G}}\boldsymbol{\vartheta} \end{aligned} \quad (75)$$

where $\tilde{\mathcal{F}} \in \mathbb{R}^{2|\mathcal{E}|m \times Nm}$ and $\tilde{\mathcal{G}} \in \mathbb{R}^{2|\mathcal{E}|m \times 2|\mathcal{E}|m}$ define the mapping between the local y variable of each robot and the slack variables $[\gamma_{ij}^T, \zeta_{ij}^T, \forall (i, j) \in \mathcal{E}]^T$ in ϑ . Note that the maximization problem in (30) has a proper, closed, and coercive objective function. Furthermore, a maximizer exists for the optimization problem in (30). As a result, our algorithm for the class of mathematical programs $\mathcal{D}$ generates bounded iterates $(\mathbf{y}^k, \vartheta^k, \varpi^k)$, which satisfy

$$\begin{aligned} & \frac{1}{\rho} \left(\|\varpi^k - \varpi^*\|_2^2 - \|\varpi^{k+1} - \varpi^*\|_2^2 \right) \\ & + \rho \left(\left\| \tilde{\mathcal{G}} (\vartheta^k - \vartheta^*) \right\|_2^2 - \left\| \tilde{\mathcal{G}} (\vartheta^{k+1} - \vartheta^*) \right\|_2^2 \right) \\ & \geq \rho \left\| \tilde{\mathcal{F}} \mathbf{y}^{k+1} - \tilde{\mathcal{G}} \vartheta^{k+1} \right\|_2^2 + \rho \left\| \tilde{\mathcal{G}} (\vartheta^{k+1} - \vartheta^*) \right\|_2^2 \end{aligned} \quad (76)$$

at each iteration, showing that the weighted error between (ϑ^k, ϖ^k) and (ϑ^*, ϖ^*)

$$\frac{1}{\rho} \|\varpi^k - \varpi^*\|_2^2 + \rho \left\| \tilde{\mathcal{G}} (\vartheta^k - \vartheta^*) \right\|_2^2 \quad (77)$$

decreases at each iteration, where $(\mathbf{y}^*, \vartheta^*, \varpi^*)$ represents a saddle point of the Lagrangian of (75).

In addition, as $k \rightarrow \infty$,

$$\begin{aligned} \tilde{\mathcal{F}} \mathbf{y}^{k+1} - \tilde{\mathcal{G}} \vartheta^{k+1} & \rightarrow 0 \quad \text{and} \\ \tilde{\mathcal{G}} (\vartheta^{k+1} - \vartheta^*) & \rightarrow 0 \end{aligned} \quad (78)$$

showing that the dual variables of all robots converge to the same solution. Ultimately, the objective value of (75) computed at $\mathbf{y}^k$ converges to its optimal value, with the iterates $(\mathbf{y}^k, \vartheta^k, \varpi^k)$ converging to a saddle point $(\mathbf{y}^*, \vartheta^*, \varpi^*)$ of the Lagrangian of (75).

APPENDIX C PROOF OF THEOREM 3

We can express the Lagrangian in (20) as

$$\mathcal{L}_a(\tilde{\mathbf{y}}, x) = \sum_{i=1}^N f_i(x_i) + \tilde{\mathbf{y}}_i^T (A_i x_i - b_i) \quad (79)$$

where robot i maintains the local dual variable $\tilde{y}_i$, with the constraints

$$\tilde{y}_i = \tilde{y}_j \quad \forall (i, j) \in \mathcal{E} \quad (80)$$

ensuring that all the dual variables have the same value. By relaxing the constraints in (80), we obtain the augmented Lagrangian

$$\begin{aligned} \mathcal{L}_a(\tilde{\mathbf{y}}, x, \gamma, \zeta, p, w) & = \sum_{i=1}^N f_i(x_i) + \tilde{\mathbf{y}}_i^T (A_i x_i - b_i) \\ & - \sum_{(i,j) \in \mathcal{E}} \left(p_{ij}^T (\tilde{y}_i - \gamma_{ij}) + w_{ij}^T (\tilde{y}_j - \zeta_{ij}) \right) \\ & - \frac{\rho}{2} \sum_{(i,j) \in \mathcal{E}} \left(\|\tilde{y}_i - \gamma_{ij}\|_2^2 + \|\tilde{y}_j - \zeta_{ij}\|_2^2 \right) \end{aligned} \quad (81)$$

with Lagrange multipliers p and w and slack variables γ and ζ for the equality constraints in (80) over the feasible set where $\gamma_{ij} = \zeta_{ij}$. From the existence of a saddle point, we can compute the optimal primal solution x^* from the problem

$$\begin{aligned} & \underset{x}{\text{minimize}} \quad \underset{\tilde{\mathbf{y}}}{\text{maximize}} \quad \mathcal{L}_a^d(\tilde{\mathbf{y}}, x, \gamma^*, \zeta^*, p^*, w^*) \\ & \text{subject to} \quad \tilde{y}_i \geq 0 \quad \forall i \in \mathcal{V} \end{aligned} \quad (82)$$

where γ^* , ζ^* , p^* , and w^* represent an optimal solution for the slack variables and Lagrange multipliers. The optimization problem in (82) simplifies to

$$\begin{aligned} & \underset{x}{\text{minimize}} \quad \underset{\tilde{\mathbf{y}}}{\text{maximize}} \quad \sum_{i=1}^N \mathcal{P}_i(\tilde{y}_i, x_i) \\ & \text{subject to} \quad \tilde{y}_i \geq 0 \quad \forall i \in \mathcal{V} \end{aligned} \quad (83)$$

where

$$\begin{aligned} \mathcal{P}_i(\tilde{y}_i, x_i) & = f_i(x_i) + \tilde{y}_i^T (A_i x_i - b_i) \\ & - r_i^{*T} \tilde{y}_i - \rho \sum_{j \in \mathcal{N}_i} \left\| \tilde{y}_i - \frac{\tilde{y}_i + \tilde{y}_j}{2} \right\|_2^2 \end{aligned} \quad (84)$$

upon convergence of the MURD-MP algorithm. The optimization problem in (83) decomposes into N independent optimization problems, with the problem for robot i given by

$$\begin{aligned} & \underset{x_i}{\text{minimize}} \quad \underset{\tilde{y}_i}{\text{maximize}} \quad \mathcal{P}_i(\tilde{y}_i, x_i) \\ & \text{subject to} \quad \tilde{y}_i \geq 0 \end{aligned} \quad (85)$$

which simplifies to the same optimization problem in (34). Hence, the local primal variables computed by all robots using (34) correspond to the optimal primal solution x^* of (18).

REFERENCES

- [1] S. Chopra, G. Notarstefano, M. Rice, and M. Egerstedt, "A distributed version of the Hungarian method for multirobot assignment," *IEEE Trans. Robot.*, vol. 33, no. 4, pp. 932–947, Aug. 2017.
- [2] J. Turner, Q. Meng, G. Schaefer, A. Whitbrook, and A. Soltoggio, "Distributed task rescheduling with time constraints for the optimization of total task allocations in a multirobot system," *IEEE Trans. Cybern.*, vol. 48, no. 9, pp. 2583–2597, Sep. 2018.
- [3] J. Scherer and B. Rinner, "Multi-robot persistent surveillance with connectivity constraints," *IEEE Access*, vol. 8, pp. 15093–15109, 2020.
- [4] L. C. B. Da Silva, R. M. Bernardo, H. A. De Oliveira, and P. F. Rosa, "Multi-UAV agent-based coordination for persistent surveillance with dynamic priorities," in *Proc. Int. Conf. Mil. Technol.*, 2017, pp. 765–771.
- [5] X. Bai, M. Cao, W. Yan, and S. S. Ge, "Efficient routing for precedence-constrained package delivery for heterogeneous vehicles," *IEEE Trans. Autom. Sci. Eng.*, vol. 17, no. 1, pp. 248–260, Jan. 2020.
- [6] N. Mathew, S. L. Smith, and S. L. Waslander, "Planning paths for package delivery in heterogeneous multirobot teams," *IEEE Trans. Autom. Sci. Eng.*, vol. 12, no. 4, pp. 1298–1308, Oct. 2015.
- [7] B. Shirani, M. Najafi, and I. Izadi, "Cooperative load transportation using multiple UAVs," *Aerosp. Sci. Technol.*, vol. 84, pp. 158–169, 2019.
- [8] D. Weyns, N. Boucké, and T. Holvoet, "Gradient field-based task assignment in an AGV transportation system," in *Proc. 5th Int. Joint Conf. Auton. Agents Multiagent Syst.*, 2006, pp. 842–849.
- [9] K. Vivaldini, L. F. Rocha, N. J. Martarelli, M. Becker, and A. P. Moreira, "Integrated tasks assignment and routing for the estimation of the optimal number of AGVS," *Int. J. Adv. Manuf. Technol.*, vol. 82, nos. 1–4, pp. 719–736, 2016.
- [10] M. Bürger, G. Notarstefano, F. Bullo, and F. Allgöwer, "A distributed simplex algorithm for degenerate linear programs and multi-agent assignments," *Automatica*, vol. 48, no. 9, pp. 2298–2304, 2012.

- [11] R. N. Haksar, O. Shorinwa, P. Washington, and M. Schwager, "Consensus-based ADMM for task assignment in multi-robot teams," in *Proc. Int. Symp. Robot. Res.*, 2019, pp. 35–51.
- [12] R. Zhang and J. Kwok, "Asynchronous distributed ADMM for consensus optimization," in *Proc. Int. Conf. Mach. Learn.*, 2014, pp. 1701–1709.
- [13] W. Deng, M.-J. Lai, Z. Peng, and W. Yin, "Parallel multi-block ADMM with $O(1/k)$ convergence," *J. Sci. Comput.*, vol. 71, no. 2, pp. 712–736, 2017.
- [14] G. Mateos, J. A. Bazerque, and G. B. Giannakis, "Distributed sparse linear regression," *IEEE Trans. Signal Process.*, vol. 58, no. 10, pp. 5262–5276, Oct. 2010.
- [15] E. Wei and A. Ozdaglar, "Distributed alternating direction method of multipliers," in *Proc. 51st IEEE Conf. Decis. Control*, 2012, pp. 5445–5450.
- [16] S. Hosseini, A. Chapman, and M. Mesbahi, "Online distributed ADMM via dual averaging," in *Proc. 53rd IEEE Conf. Decis. Control*, 2014, pp. 904–909.
- [17] L. Liu and D. A. Shell, "Optimal market-based multi-robot task allocation via strategic pricing," in *Proc. Robot.: Sci. Syst. Conf.*, 2013, vol. 9, no. 1, pp. 33–40.
- [18] H. W. Kuhn, "The Hungarian method for the assignment problem," *Nav. Res. Logistics Quart.*, vol. 2, no. 12, pp. 83–97, 1955.
- [19] S. Giordani, M. Lujak, and F. Martinelli, "A distributed algorithm for the multi-robot task allocation problem," in *Trends in Applied Intelligent Systems*, N. García-Pedrajas, F. Herrera, C. Fyfe, J. M. Benítez, and M. Ali, Eds. Berlin, Germany: Springer, 2010, pp. 721–730.
- [20] M. Berhault et al., "Robot exploration with combinatorial auctions," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2003, vol. 2, pp. 1957–1962.
- [21] D. P. Bertsekas, "The auction algorithm: A distributed relaxation method for the assignment problem," *Ann. Oper. Res.*, vol. 14, no. 1, pp. 105–123, 1988.
- [22] R. Zlot and A. Stentz, "Market-based multirobot coordination for complex tasks," *Int. J. Robot. Res.*, vol. 25, no. 1, pp. 73–101, 2006.
- [23] E. Nunes and M. Gini, "Multi-robot auctions for allocation of tasks with temporal constraints," in *Proc. AAAI Conf. Artif. Intell.*, 2015, pp. 2110–2216.
- [24] M. G. Lagoudakis, M. Berhault, S. Koenig, P. Keskinocak, and A. J. Kleywegt, "Simple auctions with performance guarantees for multi-robot task allocation," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2004, vol. 1, pp. 698–705.
- [25] S. Sariel and T. Balch, "Real time auction based allocation of tasks for multi-robot exploration problem in dynamic environments," in *Proc. AAAI Workshop Integr. Plan. Scheduling*, 2005, pp. 27–33.
- [26] T. Lemaire, R. Alami, and S. Lacroix, "A distributed tasks allocation scheme in multi-UAV context," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2004, pp. 3622–3627.
- [27] P. Sujit and R. Beard, "Distributed sequential auctions for multiple UAV task allocation," in *Proc. IEEE Amer. Control Conf.*, 2007, pp. 3955–3960.
- [28] S. L. Smith and F. Bullo, "Monotonic target assignment for robotic networks," *IEEE Trans. Autom. Control*, vol. 54, no. 9, pp. 2042–2057, Sep. 2009.
- [29] H. Choi, L. Brunet, and J. P. How, "Consensus-based decentralized auctions for robust task allocation," *IEEE Trans. Robot.*, vol. 25, no. 4, pp. 912–926, Aug. 2009.
- [30] N. Michael, M. M. Zavlanos, V. Kumar, and G. J. Pappas, "Distributed multi-robot task assignment and formation control," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2008, pp. 128–133.
- [31] L. Luo, N. Chakraborty, and K. Sycara, "Distributed algorithm design for multi-robot task assignment with deadlines for tasks," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2013, pp. 3007–3013.
- [32] T. Mercker, D. W. Casbeer, P. T. Millet, and M. R. Akella, "An extension of consensus-based auction algorithms for decentralized, time-constrained task assignment," in *Proc. Amer. Control Conf.*, 2010, pp. 6324–6329.
- [33] M. M. Zavlanos, L. Spesivtsev, and G. J. Pappas, "A distributed auction algorithm for the assignment problem," in *Proc. 47th IEEE Conf. Decis. Control*, 2008, pp. 1212–1217.
- [34] W. Yao, N. Qi, N. Wan, and Y. Liu, "An iterative strategy for task assignment and path planning of distributed multiple unmanned aerial vehicles," *Aerosp. Sci. Technol.*, vol. 86, pp. 455–464, 2019.
- [35] M. Otte, M. J. Kuhlman, and D. Sofge, "Auctions for multi-robot task allocation in communication limited environments," *Auton. Robots*, vol. 44, no. 3, pp. 547–584, 2020.
- [36] E. Schneider, E. I. Sklar, S. Parsons, and A. T. Özgelen, "Auction-based task allocation for multi-robot teams in dynamic environments," in *Proc. Conf. Towards Auton. Robot. Syst.*, 2015, pp. 246–257.
- [37] C. Schumacher, P. Chandler, M. Pachter, and L. Pachter, "UAV task assignment with timing constraints via mixed-integer linear programming," in *Proc. AIAA 3rd "Unmanned Unlimited" Tech. Conf., Workshop Exhib.*, 2004, Art. no. 6410.
- [38] M. Darrah, W. Niland, and B. Stolarik, "Multiple UAV dynamic task allocation using mixed integer linear programming in a SEAD mission," in *Proc. Infotech, Aerosp.*, 2005, Art. no. 7164.
- [39] R. J. Afonso, M. R. Maximo, and R. K. Galvão, "Task allocation and trajectory planning for multiple agents in the presence of obstacle and connectivity constraints with mixed-integer linear programming," *Int. J. Robust Nonlinear Control*, vol. 30, no. 14, pp. 5464–5491, 2020.
- [40] N. Atay and B. Bayazit, "Mixed-integer linear programming solution to multi-robot task allocation problem," Washington Univ., St Louis, MO, USA, Tech. Rep. WUCSE-2006-54, 2006.
- [41] A. Falsone, K. Margellos, and M. Prandini, "A decentralized approach to multi-agent MILPs: Finite-time feasibility and performance guarantees," *Automatica*, vol. 103, pp. 141–150, 2019.
- [42] A. Camisa, I. Notarnicola, and G. Notarstefano, "A primal decomposition method with suboptimality bounds for distributed mixed-integer linear programming," in *Proc. IEEE Conf. Decis. Control*, 2018, pp. 3391–3396.
- [43] S. Karaman and G. Inalhan, "Large-scale task/target assignment for UAV fleets using a distributed branch and price optimization scheme," *IFAC Proc. Vol.*, vol. 41, no. 2, pp. 13310–13317, 2008.
- [44] A. Testa, A. Rucco, and G. Notarstefano, "A finite-time cutting plane algorithm for distributed mixed integer linear programming," in *Proc. IEEE 56th Annu. Conf. Decis. Control*, 2017, pp. 3847–3852.
- [45] A. Settimi and L. Pallottino, "A subgradient based algorithm for distributed task assignment for heterogeneous mobile robots," in *Proc. 52nd IEEE Conf. Decis. Control*, 2013, pp. 3665–3670.
- [46] A. Khamis, A. Hussein, and A. Elmogy, "Multi-robot task allocation: A review of the state-of-the-art," *Cooperative Robots Sens. Netw.*, vol. 2015, pp. 31–51, 2015.
- [47] G. A. Korsah, A. Stentz, and M. B. Dias, "A comprehensive taxonomy for multi-robot task allocation," *Int. J. Robot. Res.*, vol. 32, no. 12, pp. 1495–1512, 2013.
- [48] J. L. Miller and L. S. Franz, "A binary-rounding heuristic for multi-period variable-task-duration assignment problems," *Comput. Oper. Res.*, vol. 23, no. 8, pp. 819–828, 1996.
- [49] J. W. Fowler, P. Wirojanagud, and E. S. Gel, "Heuristics for workforce planning with worker differences," *Eur. J. Oper. Res.*, vol. 190, no. 3, pp. 724–740, 2008.
- [50] S. Boyd, S. P. Boyd, and L. Vandenberghe, *Convex Optimization*. Cambridge, U.K.: Cambridge Univ. Press, 2004.
- [51] T. Chang, M. Hong, and X. Wang, "Multi-agent distributed optimization via inexact consensus ADMM," *IEEE Trans. Signal Process.*, vol. 63, no. 2, pp. 482–497, Jan. 2015.
- [52] *Gurobi Optimizer Reference Manual*, Gurobi Optim., LLC, Houston, TX, USA, 2022. [Online]. Available: <https://www.gurobi.com>
- [53] E. Bishop, "A generalization of the Stone-Weierstrass theorem," *Pacific J. Math.*, vol. 11, no. 3, pp. 777–783, 1961.
- [54] S. Boyd, N. Parikh, E. Chu, B. Peleato, and J. Eckstein, "Distributed optimization and statistical learning via the alternating direction method of multipliers," *Found. Trends Mach. Learn.*, vol. 3, no. 1, pp. 1–122, 2011.
- [55] L. Chen, D. Sun, and K.-C. Toh, "A note on the convergence of ADMM for linearly constrained convex optimization problems," *Comput. Optim. Appl.*, vol. 66, no. 2, pp. 327–343, 2017.



Ola Shorinwa is working toward the Ph.D. degree with the Department of Mechanical Engineering, Stanford University, Stanford, CA, USA.

He is a Member of the Multirobot Systems Lab, Stanford University. His research interests include distributed control, optimization, estimation, and planning for multirobot systems.



Ravi N. Haksar received the B.S. degree in mechanical engineering from the Georgia Institute of Technology, Atlanta, GA, USA, in 2014, and the M.S. and Ph.D. degrees in mechanical engineering from Stanford University, Stanford, CA, USA, in 2017 and 2020, respectively.

He is currently with Nuro, Mountain View, CA. His research interests include control theory, decision making under uncertainty, decentralized optimization, and cooperative multirobot teams.



Patrick Washington (Graduate Student Member, IEEE) received the B.S. degree in aerospace engineering from the University of Maryland, College Park, MD, USA, in 2017, and the M.S. degree in aeronautics and astronautics engineering in 2020 from Stanford University, Stanford, CA, USA, where he is currently working toward the Ph.D. degree with the Department of Aeronautics and Astronautics.

His research interests include task scheduling and controlling systems based on learned dynamical models.



Mac Schwager (Member, IEEE) received the B.S. degree from Stanford University, Stanford, CA, USA, in 2000, and the M.S. and Ph.D. degrees from the Massachusetts Institute of Technology (MIT), Cambridge, MA, USA, in 2005 and 2009, respectively, all in mechanical engineering.

He is currently an Associate Professor with the Department of Aeronautics and Astronautics, Stanford University. He was a Postdoctoral Researcher working jointly with the GRASP Laboratory, University of Pennsylvania, Philadelphia, PA, USA, and the Computer Science and Artificial Intelligence Laboratory, MIT, from 2010 to 2012. From 2012 to 2015, he was an Assistant Professor with Boston University, Boston, MA. His research interests include distributed algorithms for control, perception, and learning in groups of robots, and models of cooperation and competition in groups of engineered and natural agents.

Dr. Schwager received the National Science Foundation CAREER Award in 2014, the Defense Advanced Research Projects Agency Young Faculty Award in 2018, a Google Faculty Research Award in 2018, and the IROS Toshio Fukuda Young Professional Award in 2019.